Национальный исследовательский университет «Высшая школа экономики»
Московский институт электроники и математики им. А. Н. Тихонова

ОБЪЕКТНАЯ МОДЕЛЬ ИСХОДНОГО КОДА

Analitik Graph

классы папки brain

Как устроены классы, где наследование, интерфейсы, абстрактные и final-классы, как они взаимодействуют и какие алгоритмы в них заложены

Проект: Analitik Graph — аналитический сервис на Java 11 и Spring Boot 2.7

Автор: Арсений, группа БИВ255

Код: com/analytics/brain — 74 файлов, 11 пакетов

Москва, 2026

Содержание

1 Как читать проект

приставки в именах, пакеты

2 Четыре принципа ООП

инкапсуляция, наследование, абстракция, полиморфизм — с кодом

3 Схемы наследования

7 диаграмм

4 Ключевые слова

final, abstract, static, модификаторы, @Override, super, enum

5 Как классы работают вместе

5 сценариев от запроса до ответа

6 Каждый класс по отдельности

74 файлов

7 Математика и алгоритмы

16 алгоритмов простыми словами

8 Те же идеи в C++

таблица соответствий

9 Вопросы и ответы

18 вопросов для защиты

1 Как читать проект

Папка brain — это серверная часть Analitik Graph: загрузка таблиц, статистика, граф связей, анализы и ИИ-аналитик. После перестройки она разбита на 11 пакетов и 74 файлов. В каждом файле ровно один публичный тип, и его вид написан прямо в имени.

74	7 199	4 + 6	51
файлов-классов	строк Java	интерфейса + абстрактных класса	final-классов

Приставка в имени = вид класса

Начало имени	Что стоит в коде	Что это значит	Файлов
`Interface…`	`public interface X`	Контракт: список методов без реализации. Объект интерфейса создать нельзя, его реализуют классы через `implements`.	4
`Abstract…`	`public abstract class X`	Недоделанный класс: общий код для наследников плюс абстрактные методы, которые наследник обязан написать. `new` к нему не применить.	6
`Final…`	`public final class X`	Законченный класс: от него нельзя наследоваться. Это либо конечный наследник, либо набор данных, либо набор статических функций.	51
`Enum…`	`public enum X`	Перечисление: фиксированный набор объектов. У нас у перечислений есть поля, конструктор и методы.	4
`Exception…`	`class X extends … Exception`	Своё исключение, наследник стандартного исключения Java.	2
`Service…`	`@Service public class X`	Компонент Spring: объект создаёт фреймворк, ровно один на всё приложение, и передаёт в конструкторы других классов.	5
`Controller…`	`@RestController public class X`	Точка входа HTTP: методы привязаны к адресам `/api/v2/…`, ответ превращается в JSON.	2

> **Как ответить на «покажи, где классы»**
>
> Каждый файл `.java` — это класс (или интерфейс, или перечисление), и имя файла совпадает с именем типа. Открываете любую папку в `com/analytics/brain` — и по первому слову имени видно, что это за вид класса. Наследование ищется по словам `extends` и `implements` в строке объявления.

Пакеты (папки)

Пакет	За что отвечает	Файлы

`mathematics`	Статистика и математика: тесты, распределения, поправка на множественные сравнения	`AbstractAnalitikMathematicsTest`, `AbstractCorrelationTest`, `AbstractSegmentComparisonTest`, `EnumEffectScale`, `FinalAnalitikMathematics`, `FinalAnovaTest`, `FinalChiSquareTest`, `FinalMultipleComparisons`, `FinalProbabilityDistributions`, `FinalRegressionCoefficientTest`, `FinalSpearmanTest`, `FinalTwoProportionTest`, `FinalWelchTest`, `InterfaceSignificanceTest`
`analysis`	Готовые анализы для ИИ и отчётов: регрессия, ABC, тренд, аномалии, финансы, качество	`AbstractAnalitikAnalysis`, `AbstractAnomalyAnalysis`, `FinalAbcAnalysis`, `FinalDriversAnalysis`, `FinalFinanceAnalysis`, `FinalGroupAnomalyAnalysis`, `FinalQualityAnalysis`, `FinalRecordAnomalyAnalysis`, `FinalTrendAnalysis`
`formula`	Интерпретатор формул для новых колонок: парсер и дерево узлов	`EnumFormulaFunction`, `EnumFormulaOperator`, `FinalBinaryNode`, `FinalCellNode`, `FinalConstantNode`, `FinalFormulaParser`, `FinalFormulaValues`, `FinalFunctionNode`, `FinalUnaryNode`, `InterfaceFormulaNode`
`data`	Таблица, колонка, типы колонок, загрузка CSV/JSON, демо-данные	`EnumColumnType`, `FinalColumn`, `FinalCsvParser`, `FinalDataLoader`, `FinalDataset`, `FinalDemoData`, `FinalJsonParser`, `FinalRows`, `InterfaceDataParser`
`query`	Сводные таблицы, графики, фильтры, сравнение групп, воронка	`FinalChartSpec`, `FinalCondition`, `FinalGrouping`, `FinalPivotSpec`, `FinalQueryEngine`
`graph`	Узлы, связи и находки графа — то, что уходит в браузер как JSON	`FinalGraphBar`, `FinalGraphEffect`, `FinalGraphFilter`, `FinalGraphInsight`, `FinalGraphLink`, `FinalGraphNode`, `FinalGraphPayload`, `FinalGraphRef`, `FinalGraphTable`
`service`	Построение графа: обзор, связи, ветки, ключевые находки	`FinalAssociationLinks`, `FinalBranchBuilder`, `FinalGraphPaths`, `FinalInsightFinder`, `ServiceBrainGraph`
`ai`	ИИ-аналитик: чат, клиент модели, инструменты, отчёты и презентации	`FinalChatRequest`, `InterfaceAiTool`, `ServiceAiChat`, `ServiceAiClient`, `ServiceAiDocuments`, `ServiceAiTools`
`api`	REST-контроллеры и тела запросов	`AbstractApiController`, `ControllerAiApi`, `ControllerBrainApi`, `FinalCreateDatasetRequest`
`text`	Русские форматы чисел, процентов и p-значений	`FinalFormatRu`
`exception`	Собственные исключения	`ExceptionAnalitikData`, `ExceptionDatasetNotFound`

Зависимости идут сверху вниз: `api` вызывает `service` и `ai`, те — `analysis`, `query` и `formula`, а все вместе опираются на `data`, `mathematics` и `text`. Математика ничего не знает про HTTP и Spring: на вход массивы чисел, на выход объекты с результатами.

2 Четыре принципа ООП — где они в проекте

Коротко: инкапсуляция — данные закрыты и меняются через методы; наследование — класс получает поля и методы родителя; абстракция — работаем с общим типом, не зная конкретного; полиморфизм — один вызов, разное поведение у разных объектов.

Инкапсуляция

Поля помечены `private` (видны только внутри класса) и часто `final` (задаются один раз в конструкторе). Снаружи состояние не испортить — можно только вызвать методы. Пример — шкала силы эффекта: пороги спрятаны внутри, наружу торчат только методы `level` и `strength`.

Инкапсуляция в перечислении · mathematics/EnumEffectScale.java, строки 5–30

```
5   CORRELATION(0.1, 0.3, 0.5, 1.0),
6   DIFFERENCE(0.2, 0.5, 0.8, 1.2);
7
8   private final double weak;
9   private final double medium;
10  private final double strong;
11  private final double fullStrength;
12
13  EnumEffectScale(double weak, double medium, double strong, double fullStrength) {
14      this.weak = weak;
15      this.medium = medium;
16      this.strong = strong;
17      this.fullStrength = fullStrength;
18  }
19
20  public String level(double effect) {
21      double size = Math.abs(effect);
22      if (size >= strong) return "strong";
23      if (size >= medium) return "medium";
24      if (size >= weak) return "weak";
25      return "none";
26  }
27
28  public double strength(double effect) {
29      return Math.min(1, Math.abs(effect) / fullStrength);
30  }
```

Второй приём — закрытый конструктор. У `FinalFormulaParser` конструктор `private`, снаружи есть только статический метод `parse`. Так никто не создаст «полуразобранный» парсер и не полезет в его счётчик позиции `pos`.

Закрытый конструктор и фабричный метод · formula/FinalFormulaParser.java, строки 20–37

```java
private final String src;
private final FinalDataset ds;
private int pos;

private FinalFormulaParser(String src, FinalDataset ds) {
    this.src = src == null ? "" : src;
    this.ds = ds;
}

public static InterfaceFormulaNode parse(String src, FinalDataset ds) {
    FinalFormulaParser parser = new FinalFormulaParser(src, ds);
    InterfaceFormulaNode root = parser.or();
    parser.skipSpaces();
    if (parser.pos < parser.src.length()) {
        throw new ExceptionAnalitikData("Ошибка в формуле около: «" +
parser.src.substring(parser.pos) + "»");
    }
    return root;
}
```

Наследование

В проекте пять иерархий. Самая показательная — статистические тесты: у всех тестов общие поля (p, q, размер эффекта, сила) и общие методы, поэтому они вынесены в абстрактного родителя, а каждый конкретный тест добавляет только своё.

Родитель	Наследники	Что наследуется
`AbstractAnalitikMathematicsTest`	`AbstractCorrelationTest`, `AbstractSegmentComparisonTest` → 6 final-тестов	поля p, q, effect, level, strength; методы significant(), classifyEffect(), pValue(), acceptQValue()
`AbstractAnalitikAnalysis`	5 final-анализов и `AbstractAnomalyAnalysis` → 2 final-анализа	поле ds, шаблонный метод run(), помощники series() и first()
`AbstractApiController`	`ControllerBrainApi`, `ControllerAiApi`	поле brain, метод dataset(id), обработчики ошибок
`IllegalArgumentException` (Java)	`ExceptionAnalitikData`	всё поведение исключения
`NoSuchElementException` (Java)	`ExceptionDatasetNotFound`	всё поведение исключения

Наследник вызывает конструктор родителя словом `super(…)` — это всегда первая строка конструктора:

super — вызов конструктора родителя · mathematics/FinalWelchTest.java, строки 17–22

```java
public FinalWelchTest(double[] a, double[] b) {
    super("t-тест Уэлча", "g", a.length, b.length);
    meanA = FinalAnalitikMathematics.mean(a);
    meanB = FinalAnalitikMathematics.mean(b);
    boolean noBase = b.length == 0 || Math.abs(meanB) < 1e-12;
    lift = noBase ? Double.NaN : (meanA - meanB) / Math.abs(meanB);
```

А это конструктор родителя, который он вызывает · mathematics/AbstractSegmentComparisonTest.java, строки 10–15

```java
protected AbstractSegmentComparisonTest(String test, String effectName, int nA, int nB) {
    super(test, effectName);
    this.nA = nA;
    this.nB = nB;
    this.n = nA + nB;
}
```

Абстракция: абстрактные классы и интерфейсы

Абстрактный метод — это «дыра» в классе, которую обязан заполнить наследник. Например, у базового теста не написано, по какой шкале Коэна мерить силу: это решают семейства тестов.

Абстрактный метод и метод, который им пользуется · mathematics/AbstractAnalitikMathematicsTest.java, строки 26–31

```
26  protected abstract EnumEffectScale scale();
27
28  protected final void classifyEffect() {
29      level = scale().level(effect);
30      strength = scale().strength(effect);
31  }
```

Семейство «корреляции» заполняет дыру — и запечатывает её словом final · mathematics/AbstractCorrelationTest.java, строки 3–13

```
3   public abstract class AbstractCorrelationTest extends AbstractAnalitikMathematicsTest {
4
5       protected AbstractCorrelationTest(String test, String effectName) {
6           super(test, effectName);
7       }
8
9       @Override
10      protected final EnumEffectScale scale() {
11          return EnumEffectScale.CORRELATION;
12      }
13  }
```

Интерфейс — ещё более «пустой» тип: в нём только сигнатуры методов. Их у нас четыре, и каждый отделяет «что нужно» от «как сделано»:

Интерфейс	Метод контракта	Кто реализует	Кто пользуется
`InterfaceSignificanceTest`	pValue(), acceptQValue(q)	все статистические тесты	поправка Бенджамини–Хохберга
`InterfaceFormulaNode`	evaluate(row)	5 видов узлов формулы	вычисляемая колонка
`InterfaceDataParser`	parse(ds, text)	CSV- и JSON-парсер	загрузчик данных
`InterfaceAiTool`	run(ds, args, emit)	17 лямбд	ИИ-агент

Полиморфизм

Полиморфизм — когда код вызывает метод у переменной общего типа, а срабатывает версия настоящего класса объекта. Решение принимается во время работы программы (динамическая диспетчеризация). В проекте это видно в шести местах.

1. Поправка на множественные сравнения работает с любым тестом

Метод знает только интерфейс · mathematics/FinalMultipleComparisons.java, строки 28–33

```
28  public static void applyFdr(List<? extends InterfaceSignificanceTest> tests) {
29      double[] p = new double[tests.size()];
30      for (int i = 0; i < p.length; i++) p[i] = tests.get(i).pValue();
31      double[] q = benjaminiHochberg(p);
32      for (int i = 0; i < q.length; i++) tests.get(i).acceptQValue(q[i]);
33  }
```

В обзоре графа в этот список попадают вперемешку Спирмен, χ^2 и ANOVA, в ветках — Уэлч и z-тест долей, в регрессии — коэффициенты. Методу всё равно: у каждого есть `pValue()` и `acceptQValue()`.

2. Шаблонный метод в анализах

Каркас одинаковый, середина — своя у каждого · analysis/AbstractAnalitikAnalysis.java, строки 17–24

```
17  public final Map<String, Object> run() {
18      Map<String, Object> result = new LinkedHashMap<>();
19      result.put("type", "table");
20      compute(result);
21      result.put("subtitle", result.get("summary"));
22      result.put("keyCols", 1);
23      return result;
24  }
```

`run()` вызывает `compute(result)`, а какая именно версия `compute` сработает, зависит от объекта: `new FinalAbcAnalysis(…).run()` — ABC, `new FinalTrendAnalysis(…).run()` — тренд.

3. Дерево формулы

Узел не знает, что у него слева и справа · formula/FinalBinaryNode.java, строки 16–20

```
16  public Object evaluate(int row) {
17      Object a = left.evaluate(row);
18      Object b = right.evaluate(row);
19      return operator.apply(a, b);
20  }
```

`left` и `right` объявлены как `InterfaceFormulaNode`. Там может оказаться число, ячейка, функция или другая операция — у каждого своя версия `evaluate`.

4. Перечисления с поведением

Каждая константа — свой анонимный подкласс со своим apply · formula/EnumFormulaOperator.java, строки 76–82

```
76  DIVIDE {
77      @Override
78      public Object apply(Object a, Object b) {
79          double divisor = number(b);
80          return divisor == 0 ? Double.NaN : number(a) / divisor;
81      }
82  },
```

5. Описание сегмента без instanceof

Один вызов — разный текст у Уэлча и у z-теста долей · service/FinalBranchBuilder.java, строки 554–555

```
554 n.sub = t.leafSubtitle(m.name, hasRest) + stars;
555 n.stats.addAll(t.leafStats(m.name, mt.segment, mt.others, hasRest));
```

6. Словарь ИИ-инструментов вместо switch

Инструмент выбирается по имени из словаря · ai/ServiceAiTools.java, строки 46–51

```
46  String run(FinalDataset ds, String name, JsonNode args, BiConsumer<String, Object> emit) throws
    Exception {
47      if (ds == null) throw new IllegalStateException("Нет загруженных данных");
48      InterfaceAiTool tool = tools.get(name);
49      if (tool == null) throw new ExceptionAnalitikData("Неизвестный инструмент " + name);
50      return tool.run(ds, args, emit);
51  }
```

Перегрузка и переопределение — не путать

Переопределение (override) — наследник пишет свою версию метода родителя с той же сигнатурой, помечает `@Override`; выбор во время выполнения. **Перегрузка (overload)** — несколько методов с одним именем и разными параметрами в одном классе, например два конструктора `FinalGraphTable()` и `FinalGraphTable(String title, String... head)`; выбор при компиляции.

3 Схемы наследования

Цвет рамки = вид класса: синий — интерфейс, оранжевый — абстрактный, зелёный — final, фиолетовый — перечисление, серый — Spring-компонент, красный — исключение.

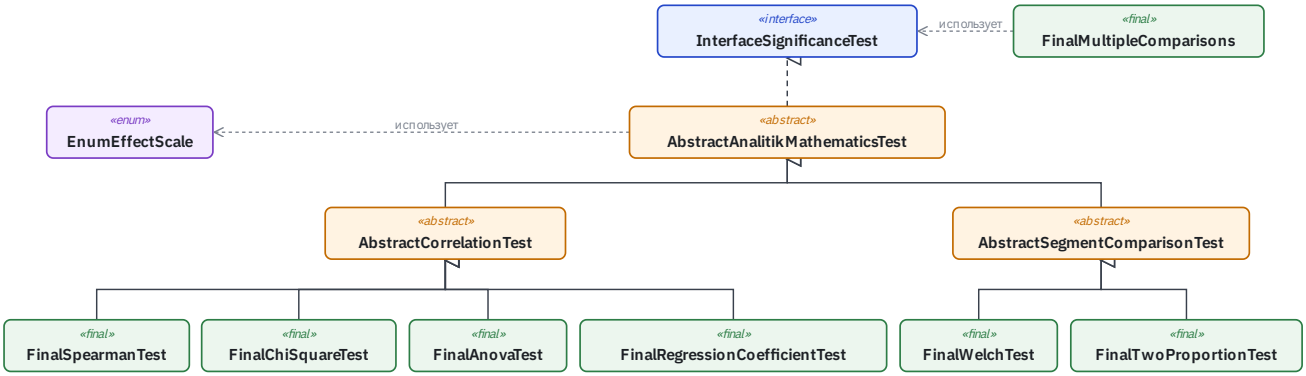


Рис. 1. Статистические тесты: интерфейс → абстрактная база → два абстрактных семейства → конечные (final) тесты. Сплошная линия с треугольником — extends, пунктир с треугольником — implements, тонкий пунктир со стрелкой — «использует».

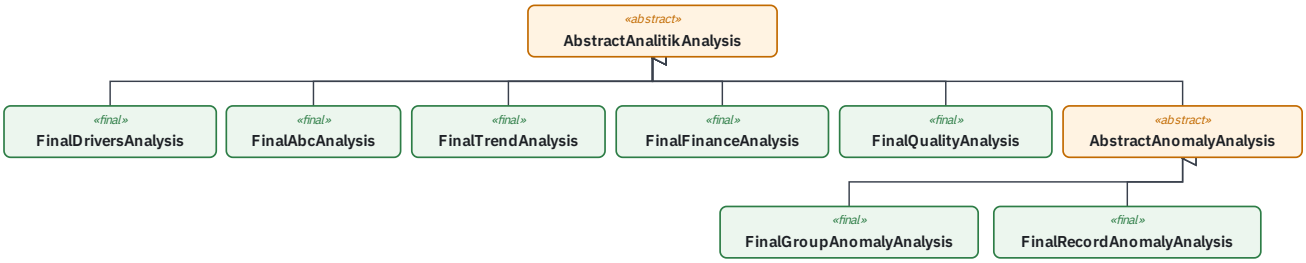


Рис. 2. Анализы: шаблонный метод run() в базе, compute() — в каждом наследнике. Аномалии — третий уровень наследования.

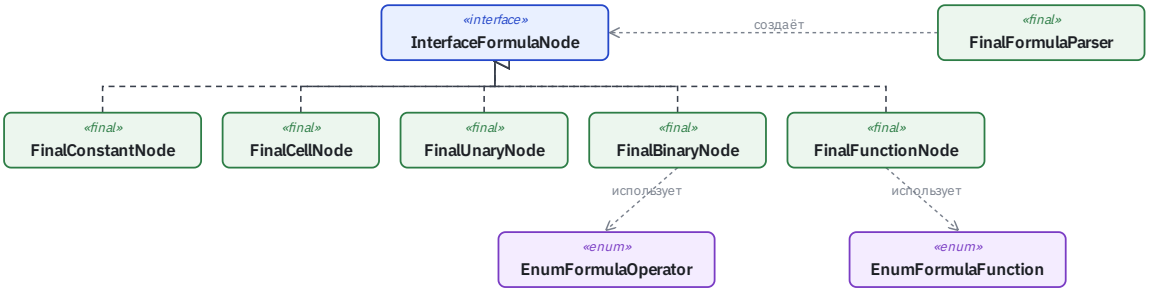


Рис. 3. Формулы: пять видов узлов дерева реализуют один интерфейс; операции и функции — перечисления с поведением.

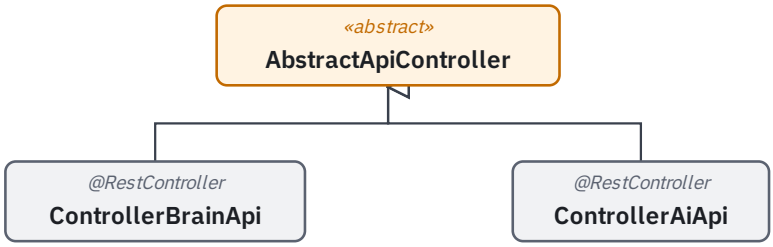


Рис. 4. Контроллеры: общий код обработки ошибок и поиска набора — в абстрактном родителе.

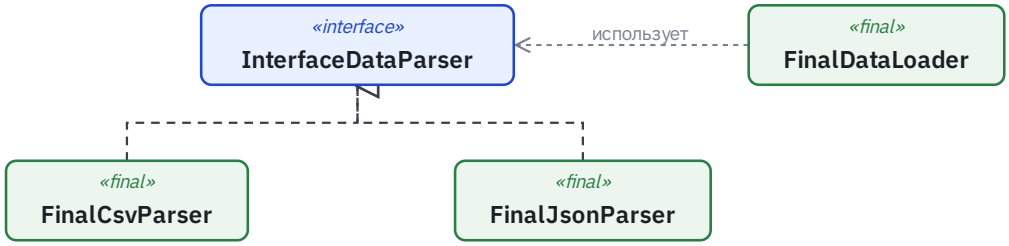


Рис. 5. Разбор данных: загрузчик выбирает парсер и работает с ним через интерфейс.

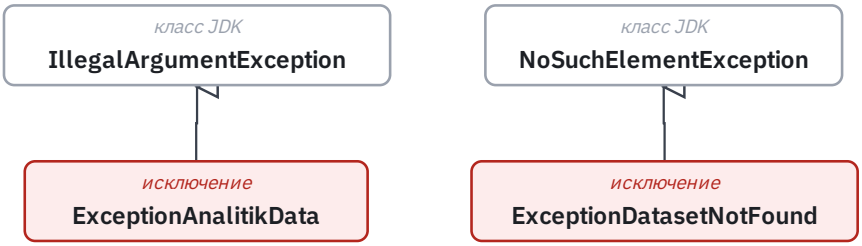


Рис. 6. Собственные исключения наследуют стандартные классы Java.

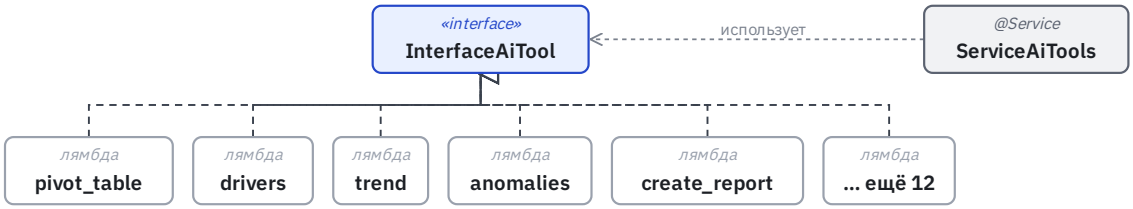


Рис. 7. ИИ-инструменты: 17 лямбд реализуют функциональный интерфейс; ServiceAiTools хранит их в словаре «имя → инструмент».

4 Ключевые слова: final, abstract, static и другие

final — три разных смысла

Где стоит	Что запрещает	Где в проекте и зачем
`final class X`	наследоваться от класса	51 класс с приставкой Final. Конечные тесты (`FinalWelchTest`) — формула теста закончена, подменять её в наследнике нельзя. Утилиты (`FinalAnalitikMathematics`) — наследовать набор статических функций бессмысленно. Классы данных (`FinalGraphNode`) — это просто контейнеры для JSON.
метод `final`	переопределять метод в наследнике	`run()` в `AbstractAnalitikAnalysis` — порядок шагов общий для всех анализов, наследник меняет только `compute()`. `significant()`, `pValue()`, `acceptQValue()`, `classifyEffect()` в базовом тесте — правило значимости одно на весь проект. `scale()` в семействах тестов — каждое семейство навсегда закрепляет свою шкалу. `compute()` в `AbstractAnomalyAnalysis` — общий каркас поиска аномалий, наследник пишет только `findAnomalies()`. `dataset(id)` в `AbstractApiController`.
поле или переменная `final`	присваивать значение второй раз	`FinalColumn.name` — имя колонки не меняется после создания. Все поля узлов формулы (`left`, `right`, `operator`) — дерево неизменяемо, его можно спокойно вычислять для тысяч строк. Поля-зависимости в сервисах (`private final ServiceAiClient client`) — Spring передаёт их один раз в конструкторе.

`static final` — константа: одно значение на весь класс, которое нельзя изменить. Например `ALPHA = 0.05` — уровень значимости, `Z_95 = 1.959964` — квантиль нормального распределения для 95% интервала, `TUKEY_K = 3` — множитель в правиле Тьюки.

Константа и final-метод, который ею пользуется · mathematics/AbstractAnalitikMathematicsTest.java, строки 5–5

```
5  public static final double ALPHA = 0.05;
```

mathematics/AbstractAnalitikMathematicsTest.java, строки 33–35

```
33  public final boolean significant() {
34      return q < ALPHA && !"none".equals(level);
35  }
```

> **Тонкость, которую любят спрашивать**
>
> `final List<String> list` запрещает присвоить переменной другой список, но элементы списка менять можно. final замораживает ссылку, а не сам объект. Поэтому в `FinalGraphTable` поле `rows` можно пополнять методом `row(…)`.

abstract

`abstract class` — класс, объект которого создать нельзя (`new AbstractAnalitikAnalysis(ds)` не скомпилируется). `abstract`-метод не имеет тела и заканчивается точкой с запятой; первый не-абстрактный наследник обязан его написать. Абстрактный класс, в отличие от интерфейса, может хранить поля и иметь конструктор — поэтому общий код тестов и анализов лежит именно в абстрактных классах.

static

`static`-метод принадлежит классу, а не объекту: вызывается как `FinalAnalitikMathematics.mean(x)`, без `new`. У него нет `this`, поэтому он не видит обычных полей. Все математические функции — статические: им не нужно состояние, только входные массивы. Статические вложенные классы (`MetricTest`, `SegmentFinding`, `Factor`) — это маленькие «записи» внутри одного файла, которые снаружи не нужны.

private / protected / public / без модификатора

Модификатор	Кто видит	Пример
`private`	только сам класс	поля узлов формулы, `correctedCramersV` в χ^2
без модификатора	классы того же пакета	вспомогательные методы `FinalGraphPaths` для классов пакета service
`protected`	пакет и наследники	конструкторы абстрактных классов, `compute()`, `scale()`, `ds`
`public`	все	поля результатов тестов (уходят в JSON), методы API

@Override, super, this

`@Override` — подсказка компилятору «этот метод переопределяет родительский». Если ошибиться в имени или параметрах, будет ошибка компиляции, а не тихий новый метод. `super(…)` — вызов конструктора родителя, `super.x` — обращение к члену родителя. `this` — ссылка на текущий объект; `this.nA = nA` отличает поле от параметра с тем же именем.

enum с полями и методами

В Java перечисление — полноценный класс. У `EnumColumnType` есть поля (русское название, анализируется ли, можно ли по нему резать), конструктор и методы. У `EnumFormulaOperator` и `EnumFormulaFunction` у каждой константы своё тело метода — это анонимные подклассы перечисления.

data/EnumColumnType.java — файл целиком

```java
package com.analytics.brain.data;

public enum EnumColumnType {

    NUMERIC("число", true, false),
    CATEGORICAL("категория", true, true),
    DATE("дата", true, true),
    ID("идентификатор", false, false),
    TEXT("текст", false, false);

    private final String russianName;
    private final boolean analyzable;
    private final boolean dimension;

    EnumColumnType(String russianName, boolean analyzable, boolean dimension) {
        this.russianName = russianName;
        this.analyzable = analyzable;
        this.dimension = dimension;
    }

    public String russianName() {
        return russianName;
    }

    public boolean analyzable() {
        return analyzable;
    }

    public boolean dimension() {
        return dimension;
    }
}
```

Вложенные классы, лямбды, ссылки на методы

Статический вложенный класс — класс внутри класса, без доступа к полям внешнего объекта: `FinalDriversAnalysis.Factor`, `FinalBranchBuilder.MetricTest`, `FinalInsightFinder.SegmentFinding`. **Лямбда** `(ds, args, emit) -> …` — короткая запись объекта, реализующего функциональный интерфейс (интерфейс с одним методом). **Ссылка на метод** `this::createReport` — то же самое, но вместо тела указан готовый метод.

5 Как классы работают вместе

Пять сценариев от запроса браузера до ответа. В каждой строке — какой объект и какой метод сейчас работает.

Сценарий 1. Загрузка CSV-файла

№	Класс и метод	Что происходит
1	`ControllerBrainApi.create`	Spring превращает JSON-тело запроса в объект `FinalCreateDatasetRequest` и вызывает метод.
2	`new FinalDataset()`	Создаётся пустой набор данных со случайным id из 8 символов.
3	`new FinalCsvParser().parse`	Парсер угадывает разделитель (запятая, точка с запятой, табуляция или |) по заголовку и читает строки.
4	`FinalDataLoader.build`	Для каждой колонки вызывается `infer`: определяется `EnumColumnType`, числа, категории или даты.
5	`FinalDataLoader.detectKpis / detectFunnel`	По словам в названиях находятся ключевые метрики и колонка этапов воронки.
6	`ServiceBrainGraph.store.put`	Набор кладётся в словарь в памяти сервера.
7	`FinalDataset.summary`	Короткое описание набора уходит в браузер как JSON.

Точка входа загрузки · api/ControllerBrainApi.java, строки 52–80

```
52  @PostMapping("/datasets")
53  public Map<String, Object> create(@RequestBody FinalCreateDatasetRequest r) throws Exception {
54      FinalDataset ds = new FinalDataset();
55      ds.id = UUID.randomUUID().toString().substring(0, 8);
56      ds.fetchedAt = System.currentTimeMillis();
57
58      if (r.url != null && !r.url.isBlank()) {
59          ds.sourceType = "url";
60          ds.sourceUrl = r.url.trim();
61          if (r.headers != null) ds.headers.putAll(r.headers);
62          String[] response = FinalDataLoader.fetch(ds.sourceUrl, ds.headers);
63          FinalDataLoader.parseAuto(ds, response[0], response[1]);
64          ds.name = r.name != null && !r.name.isBlank() ? r.name : shortName(ds.sourceUrl);
65      } else if (r.data != null && !r.data.isNull()) {
66          ds.sourceType = "push";
67          new FinalJsonParser().parse(ds, r.data.toString());
68          ds.name = r.name != null ? r.name : "API-данные";
69      } else if (r.csv != null && !r.csv.isBlank()) {
70          ds.sourceType = "file";
71          new FinalCsvParser().parse(ds, r.csv);
72          ds.name = r.name != null ? r.name : "CSV";
73      } else {
74          throw new ExceptionAnalitikData("Передайте csv, data или url");
75      }
76
77      FinalDataLoader.build(ds);
78      brain.store.put(ds.id, ds);
79      return ds.summary();
80  }
```

Сценарий 2. Обзорный граф

№	Класс и метод	Что происходит

1	`ControllerBrainApi.overview`	Метод `dataset(id)`, унаследованный от `AbstractApiController`, находит набор или бросает `ExceptionDatasetNotFound` (ответ 404).
2	`ServiceBrainGraph.overview`	Если данные не менялись, отдаётся кэш. Иначе строится центр и по узлу на каждую переменную (`FinalBranchBuilder.metricNode / dimNode`).
3	`FinalAssociationLinks.association`	Для каждой пары переменных выбирается тест по типам колонок и создаётся объект теста.
4	`new FinalSpearmanTest / FinalChiSquareTest / FinalAnovaTest`	Конструктор теста сразу всё считает: статистику, p, размер эффекта, силу.
5	`FinalMultipleComparisons.applyFdr`	Поправка Бенджамини–Хохберга проставляет каждому тесту q-значение.
6	`AbstractAnalitikMathematicsTest.significant`	На граф попадают только связи с q < 0,05 и силой не ниже «слабой».
7	`FinalInsightFinder.insights`	Ищутся «ключевые находки» для левой панели.
8	`FinalGraphPayload → JSON`	Jackson превращает публичные поля объектов графа в JSON.

Сердце обзора: все пары, поправка, отбор · service/ServiceBrainGraph.java, строки 47–66

```
47  List<FinalGraphLink> pairs = new ArrayList<>();
48  List<AbstractAnalitikMathematicsTest> tests = new ArrayList<>();
49  for (int i = 0; i < vars.size(); i++) {
50      for (int j = i + 1; j < vars.size(); j++) {
51          FinalGraphLink link = FinalAssociationLinks.association(vars.get(i), vars.get(j));
52          if (link != null) {
53              pairs.add(link);
54              tests.add(link.test);
55          }
56      }
57  }
58  FinalMultipleComparisons.applyFdr(tests);
59
60  for (FinalGraphLink link : pairs) {
61      if (!link.test.significant()) continue;
62      link.level = link.test.level;
63      link.strength = link.test.strength;
64      link.text = link.text + " q " + fmtPq(link.test.q) + " (с поправкой на множественные
    сравнения).";
65      p.links.add(link);
66  }
```

Сценарий 3. Раскрытие ветки «Подписка = Премиум»

№	Класс и метод	Что происходит
1	`ControllerBrainApi.expand`	Тело запроса превращается в `FinalGraphRef`: вид узла и путь фильтров до него.
2	`ServiceBrainGraph.expand`	По виду узла (`dim / cat / more / metric / funnel`) выбирается метод раскрытия.
3	`FinalRows.matching / restOf`	Номера строк сегмента и «остальных внутри родителя».
4	`FinalBranchBuilder.addMetricLeaves`	Для каждой числовой метрики сегмент сравнивается с остальными.
5	`AbstractSegmentComparisonTest.compare`	Фабричный метод: для «счётчиков» создаёт `FinalTwoProportionTest`, для остальных чисел — `FinalWelchTest`.
6	`FinalBranchBuilder.metricLeaf`	Лист графа описывается полиморфно: `leafSubtitle`, `leafStats`, `statisticSymbol`.

Фабричный метод: возвращает общий тип, создаёт конкретный · mathematics/AbstractSegmentComparisonTest.java, строки 17–20

```
17  public static AbstractSegmentComparisonTest compare(boolean countLike, double[] segment,
    double[] others) {
18      if (countLike) return FinalTwoProportionTest.ofPositive(segment, others);
19      return new FinalWelchTest(segment, others);
20  }
```

Сценарий 4. Вопрос ИИ-аналитику

№	Класс и метод	Что происходит
1	`ControllerAiApi.chat`	Открывается поток SSE; разговор идёт в отдельном потоке из пула, чтобы не держать сервер.
2	`ServiceAiChat.chat`	Собирает сообщения и в цикле (не больше 10 шагов) обращается к модели.
3	`ServiceAiClient.chatCompletion`	HTTP-запрос к модели по OpenAI-совместимому API.
4	`ServiceAiChat.runCall → ServiceAiTools.run`	Если модель попросила инструмент, он берётся из словаря `Map<String, InterfaceAiTool>`.
5	`InterfaceAiTool.run (лямбда)`	Например, `new FinalDriversAnalysis(…).run()` — регрессия на настоящих данных.
6	`emit("artifact", …)`	Таблица сразу уходит в браузер, а модель получает сжатую версию и пишет ответ.

Сценарий 5. Новая колонка по формуле

№	Класс и метод	Что происходит
1	`ControllerAiApi.column`	Получает имя и текст формулы, например `if([NPS] >= 9, "Промоутер", "Прочие")`.
2	`FinalQueryEngine.addColumn`	Просит парсер построить дерево.
3	`FinalFormulaParser.parse`	Рекурсивный спуск превращает текст в дерево объектов `InterfaceFormulaNode`.
4	`InterfaceFormulaNode.evaluate(i)`	Дерево вычисляется для каждой строки: каждый узел спрашивает значения у своих детей.
5	`FinalDataLoader.build`	Колонки пересобираются, версия набора растёт — кэш обзора сбрасывается.

Дерево строится один раз и вычисляется для каждой строки · query/FinalQueryEngine.java, строки 739–750

```
739  InterfaceFormulaNode f = FinalFormulaParser.parse(formula, ds);
740
741  List<String> values = new ArrayList<>();
742  for (int i = 0; i < ds.rows; i++) {
743      Object v;
744      try {
745          v = f.evaluate(i);
746      } catch (ArithmeticException e) {
747          v = null;
748      }
749      values.add(cellText(v));
750  }
```

6 Каждый класс по отдельности

По пакетам, в порядке «от общего к частному»: сначала интерфейсы и абстрактные классы, потом их наследники, потом утилиты. У каждого файла — вид, связи, объявление, зачем он нужен, почему он именно такого вида и таблица методов с номерами строк.

6.1. Пакет mathematics — статистика и математика

6.1.1 InterfaceSignificanceTest `mathematics/InterfaceSignificanceTest.java · 8 строк`

Интерфейс наследники: `AbstractAnalitikMathematicsTest`

```
public interface InterfaceSignificanceTest
```

Контракт «у меня есть p-значение, и мне можно выставить q». Больше поправке на множественные сравнения ничего не нужно.

> **Почему такой вид.** Интерфейс, потому что поправку нужно применять к очень разным объектам: к тестам связей, сравнению сегментов, коэффициентам регрессии. Общее у них только эти два действия.

mathematics/InterfaceSignificanceTest.java — файл целиком

```java
package com.analytics.brain.mathematics;

public interface InterfaceSignificanceTest {

    double pValue();

    void acceptQValue(double q);
}
```

6.1.2 AbstractAnalitikMathematicsTest `mathematics/AbstractAnalitikMathematicsTest.java · 46 строк`

Абстрактный класс реализует `InterfaceSignificanceTest` · наследники: `AbstractCorrelationTest`, `AbstractSegmentComparisonTest`

```
public abstract class AbstractAnalitikMathematicsTest implements InterfaceSignificanceTest
```

Общий предок всех статистических тестов. Хранит то, что есть у любого теста: название, статистику, степени свободы, p и q, размер эффекта с интервалом, объём выборки, словесную силу связи и число 0..1 для толщины линии на графе.

> **Почему такой вид.** Абстрактный: «тест вообще» не существует, у него нет формулы. Поля публичные, потому что объект целиком уходит в браузер как JSON (Jackson берёт публичные поля).

Стр.	Объявление	Что делает
21	`protected AbstractAnalitikMathematicsTest(String test, String effectName)`	protected-конструктор: вызывается только наследниками через super
26	`protected abstract EnumEffectScale scale();`	abstract: какой шкалой Коэна мерить силу — решают семейства
28	`protected final void classifyEffect()`	final: переводит размер эффекта в «слабая/средняя/сильная» и в толщину линии
33	`public final boolean significant()`	final: q < 0,05 и сила не «none» — единое правило для всего проекта
38	`public final double pValue()`	final: реализация интерфейса
43	`public final void acceptQValue(double q)`	final: реализация интерфейса

Базовый класс целиком · mathematics/AbstractAnalitikMathematicsTest.java, строки 3–46

```java
public abstract class AbstractAnalitikMathematicsTest implements InterfaceSignificanceTest {

    public static final double ALPHA = 0.05;

    public String test;
    public double stat;
    public double df1 = Double.NaN;
    public double df2 = Double.NaN;
    public double p = 1;
    public double q = 1;
    public String effectName;
    public double effect;
    public double lo = Double.NaN;
    public double hi = Double.NaN;
    public int n;
    public String level = "none";
    public double strength;

    protected AbstractAnalitikMathematicsTest(String test, String effectName) {
        this.test = test;
        this.effectName = effectName;
    }

    protected abstract EnumEffectScale scale();

    protected final void classifyEffect() {
        level = scale().level(effect);
        strength = scale().strength(effect);
    }

    public final boolean significant() {
        return q < ALPHA && !"none".equals(level);
    }

    @Override
    public final double pValue() {
        return p;
    }

    @Override
    public final void acceptQValue(double q) {
        this.q = q;
    }
}
```

6.1.3 AbstractCorrelationTest `mathematics/AbstractCorrelationTest.java · 13 строк`

Абстрактный класс наследует `AbstractAnalitikMathematicsTest` · наследники: `FinalAnovaTest`, `FinalChiSquareTest`, `FinalRegressionCoefficientTest`, `FinalSpearmanTest`

```
public abstract class AbstractCorrelationTest extends AbstractAnalitikMathematicsTest
```

Семейство тестов «насколько связаны две переменные»: ρ Спирмена, V Крамера, η, стандартизованный β.

> **Почему такой вид.** Абстрактный — сам по себе не считает ничего; нужен, чтобы один раз закрепить шкалу 0,1 / 0,3 / 0,5 за всей группой. Метод scale() здесь final.

Стр.	Объявление	Что делает
5	`protected AbstractCorrelationTest(String test, String effectName)`	конструктор

10	`protected final EnumEffectScale scale()`	final: всегда EnumEffectScale.CORRELATION

6.1.4 AbstractSegmentComparisonTest `mathematics/AbstractSegmentComparisonTest.java · 40 строк`

Абстрактный класс наследует `AbstractAnalitikMathematicsTest` · наследники: `FinalTwoProportionTest`, `FinalWelchTest`

```
public abstract class AbstractSegmentComparisonTest extends AbstractAnalitikMathematicsTest
```

Семейство тестов «сегмент против остальных»: Уэлч для средних и z-тест для долей. Добавляет размеры групп nA и nB и методы, которые описывают результат словами.

> **Почему такой вид.** Абстрактный: у Уэлча и z-теста разные формулы и разный текст, но BrainBuilder должен работать с ними одинаково. Поэтому здесь объявлены абстрактные методы описания, а в конце стоит статический фабричный метод compare.

Стр.	Объявление	Что делает
10	`protected AbstractSegmentComparisonTest(String test, String effectName, int nA, int nB)`	конструктор
17	`public static AbstractSegmentComparisonTest compare(boolean countLike, double[] segment, double[] others)`	static: фабрика — выбирает Уэлча или z-тест и возвращает общий тип
23	`protected final EnumEffectScale scale()`	final: всегда EnumEffectScale.DIFFERENCE (шкала 0,2 / 0,5 / 0,8)
27	`public final boolean effectIntervalShown()`	final: показывать ли интервал эффекта в карточке
31	`protected abstract boolean intervalDescribesEffect();`	abstract: интервал lo..hi относится к эффекту (Уэлч) или к разнице долей (z-тест)
33	`public abstract String statisticSymbol();`	abstract: «t = » или «z = »
35	`public abstract String leafSubtitle(String metric, boolean hasRest);`	abstract: подпись листа графа
37	`public abstract List<Object[]> leafStats(String metric, double[] segment, double[] others, boolean ha…`	abstract: строки карточки
39	`public abstract String insightText(String metric);`	abstract: текст «ключевой находки»

Абстрактные методы семейства · mathematics/AbstractSegmentComparisonTest.java, строки 5–40

```java
public abstract class AbstractSegmentComparisonTest extends AbstractAnalitikMathematicsTest {

    public int nA;
    public int nB;

    protected AbstractSegmentComparisonTest(String test, String effectName, int nA, int nB) {
        super(test, effectName);
        this.nA = nA;
        this.nB = nB;
        this.n = nA + nB;
    }

    public static AbstractSegmentComparisonTest compare(boolean countLike, double[] segment, double[] others) {
        if (countLike) return FinalTwoProportionTest.ofPositive(segment, others);
        return new FinalWelchTest(segment, others);
    }

    @Override
    protected final EnumEffectScale scale() {
        return EnumEffectScale.DIFFERENCE;
    }

    public final boolean effectIntervalShown() {
        return !Double.isNaN(lo) && intervalDescribesEffect();
    }

    protected abstract boolean intervalDescribesEffect();

    public abstract String statisticSymbol();

    public abstract String leafSubtitle(String metric, boolean hasRest);

    public abstract List<Object[]> leafStats(String metric, double[] segment, double[] others, boolean hasRest);

    public abstract String insightText(String metric);
}
```

6.1.5 FinalSpearmanTest `mathematics/FinalSpearmanTest.java · 26 строк`

Final-класс наследует `AbstractCorrelationTest`

```
public final class FinalSpearmanTest extends AbstractCorrelationTest
```

Корреляция Спирмена двух числовых колонок (или даты и числа) с 95% интервалом.

> **Почему такой вид.** Final: формула закончена. Всё считается в конструкторе — после new объект уже содержит результат.

Стр.	Объявление	Что делает
5	`public FinalSpearmanTest(double[] a, double[] b)`	конструктор: пары без пропусков → ранги → Пирсон по рангам → t и p → интервал Фишера

Весь алгоритм · mathematics/FinalSpearmanTest.java, строки 5–25

```
5   public FinalSpearmanTest(double[] a, double[] b) {
6       super("Корреляция Спирмена", "ρ");
7       double[][] pairs = FinalAnalitikMathematics.bothPresent(a, b);
8       double[] x = pairs[0];
9       double[] y = pairs[1];
10      n = x.length;
11      if (n < 5) return;
12
13      double rho = FinalAnalitikMathematics.pearson(FinalAnalitikMathematics.ranks(x),
    FinalAnalitikMathematics.ranks(y));
14      rho = Math.max(-0.999999, Math.min(0.999999, rho));
15      effect = rho;
16      stat = rho * Math.sqrt((n - 2) / (1 - rho * rho));
17      df1 = n - 2;
18      p = FinalProbabilityDistributions.tP(stat, n - 2);
19
20      double z = 0.5 * Math.log((1 + rho) / (1 - rho));
21      double se = Math.sqrt(1.06 / Math.max(1, n - 3));
22      lo = Math.tanh(z - 1.96 * se);
23      hi = Math.tanh(z + 1.96 * se);
24      classifyEffect();
25  }
```

6.1.6 FinalChiSquareTest `mathematics/FinalChiSquareTest.java · 80 строк`

Final-класс наследует `AbstractCorrelationTest`

```
public final class FinalChiSquareTest extends AbstractCorrelationTest
```

χ²-тест независимости двух категорий и исправленный V Крамера. Хранит таблицы наблюдаемых и ожидаемых частот.

> **Почему такой вид.** Final. Поля observed/expected/rows/cols публичные — браузер рисует по ним подсказку.

Стр.	Объявление	Что делает
15	`public FinalChiSquareTest(String[] a, String[] b)`	строит таблицу сопряжённости, ожидаемые частоты, χ², df, p, V
68	`private double correctedCramersV(double chi2, int r, int k)`	private: V с поправкой Бергсмы на маленькие выборки
77	`public double residual(int row, int col)`	стандартизованный остаток клетки: где пар больше, чем при независимости

6.1.7 FinalAnovaTest `mathematics/FinalAnovaTest.java · 79 строк`

Final-класс наследует `AbstractCorrelationTest`

```
public final class FinalAnovaTest extends AbstractCorrelationTest
```

Однофакторный дисперсионный анализ (категория × число) плюс Краскел–Уоллис по рангам.

> **Почему такой вид.** Final. Краскел–Уоллис вынесен в private-метод, чтобы конструктор читался по шагам.

Стр.	Объявление	Что делает
16	`public FinalAnovaTest(String[] groupOf, double[] y)`	группирует значения, считает межгрупповую и внутригрупповую суммы квадратов, F, p, η², ω²

66	`private void kruskalWallis(double[] all, double dfBetween)`	private: H-статистика по рангам и её p

6.1.8 FinalRegressionCoefficientTest `mathematics/FinalRegressionCoefficientTest.java · 16 строк`

Final-класс наследует `AbstractCorrelationTest`

```
public final class FinalRegressionCoefficientTest extends AbstractCorrelationTest
```

t-тест одного коэффициента регрессии: t = β / SE.

> **Почему такой вид.** Final и маленький: нужен, чтобы коэффициенты регрессии тоже прошли через общую поправку Бенджамини–Хохберга как обычные тесты.

Стр.	Объявление	Что делает
8	`public FinalRegressionCoefficientTest(String factor, double beta, double standardError, int degreesOfFreedom)`	конструктор

6.1.9 FinalWelchTest `mathematics/FinalWelchTest.java · 85 строк`

Final-класс наследует `AbstractSegmentComparisonTest`

```
public final class FinalWelchTest extends AbstractSegmentComparisonTest
```

t-тест Уэлча (средние двух групп, разброс может быть разный) и g Хеджеса.

> **Почему такой вид.** Final-наследник AbstractSegmentComparisonTest; реализует все абстрактные методы описания.

Стр.	Объявление	Что делает
17	`public FinalWelchTest(double[] a, double[] b)`	средние, lift, t, степени свободы Уэлча–Саттертуэйта, p, g Хеджеса с интервалом
51	`protected boolean intervalDescribesEffect()`	true
56	`public String statisticSymbol()`	«t = »
61	`public String leafSubtitle(String metric, boolean hasRest)`	«x̄ 123 · +45%»
67	`public List<Object[]> leafStats(String metric, double[] segment, double[] others, boolean hasRest)`	среднее с интервалом, медиана, среднее остальных, относительная разница
80	`public String insightText(String metric)`	«Выручка выше на 195% (… g = 0,85, q < 0,001)»

6.1.10 FinalTwoProportionTest `mathematics/FinalTwoProportionTest.java · 86 строк`

Final-класс наследует `AbstractSegmentComparisonTest`

```
public final class FinalTwoProportionTest extends AbstractSegmentComparisonTest
```

z-тест разности двух долей и h Коэна. Используется для «счётчиков»: доля тех, у кого значение больше нуля.

> **Почему такой вид.** Final-наследник того же семейства, что и Уэлч, но с другой формулой и другим текстом.

Стр.	Объявление	Что делает
19	`public FinalTwoProportionTest(int kA, int nA, int kB, int nB)`	доли pA и pB, объединённая доля, z, p, интервал разницы, h = 2·arcsin√pA − 2·arcsin√pB
39	`public static FinalTwoProportionTest ofPositive(double[] segment, double[] others)`	static: фабрика «доля значений больше нуля»
43	`public static int countPositive(double[] values)`	static: сколько значений больше нуля
52	`protected boolean intervalDescribesEffect()`	false
57	`public String statisticSymbol()`	«z = »
62	`public String leafSubtitle(String metric, boolean hasRest)`	«купили 42% · +20,8 п.п.»
67	`public List<Object[]> leafStats(String metric, double[] segment, double[] others, boolean hasRest)`	доля с интервалом Уилсона, среднее, доля у остальных, разница, «во сколько раз чаще»
82	`public String insightText(String metric)`	«купили 42% против 22% у остальных (...)»

6.1.11 EnumEffectScale `mathematics/EnumEffectScale.java · 31 строк`

Перечисление

`public enum EnumEffectScale`

Две шкалы силы эффекта по Коэну: для коэффициентов связи и для разниц между группами.

> **Почему такой вид.** Enum: шкал ровно две и новых не появится. У каждой константы свои пороги — они передаются в конструктор перечисления.

Константы: `CORRELATION`, `DIFFERENCE`

mathematics/EnumEffectScale.java — файл целиком

```java
package com.analytics.brain.mathematics;

public enum EnumEffectScale {

    CORRELATION(0.1, 0.3, 0.5, 1.0),
    DIFFERENCE(0.2, 0.5, 0.8, 1.2);

    private final double weak;
    private final double medium;
    private final double strong;
    private final double fullStrength;

    EnumEffectScale(double weak, double medium, double strong, double fullStrength) {
        this.weak = weak;
        this.medium = medium;
        this.strong = strong;
        this.fullStrength = fullStrength;
    }

    public String level(double effect) {
        double size = Math.abs(effect);
        if (size >= strong) return "strong";
        if (size >= medium) return "medium";
        if (size >= weak) return "weak";
        return "none";
    }

    public double strength(double effect) {
        return Math.min(1, Math.abs(effect) / fullStrength);
    }
}
```

6.1.12 FinalAnalitikMathematics `mathematics/FinalAnalitikMathematics.java · 243 строк`

Final-класс

`public final class FinalAnalitikMathematics`

Главная «математичка» проекта: описательная статистика и вспомогательные алгоритмы — всё, что не является отдельным тестом.

> **Почему такой вид.** Final-класс со статическими методами и закрытым конструктором: объект не нужен, наследовать нечего. Аналог пространства имён (namespace) в C++.

Стр.	Объявление	Что делает
13	`private FinalAnalitikMathematics()`	конструктор
16	`public static double[] clean(double[] x)`	убрать NaN (пропуски)
29	`public static double mean(double[] x)`	среднее
36	`public static double variance(double[] x)`	выборочная дисперсия (делим на n − 1)
44	`public static double sd(double[] x)`	стандартное отклонение
48	`public static double[] sorted(double[] x)`	отсортированная копия
54	`public static double quantile(double[] sorted, double p)`	квантиль с линейной интерполяцией
62	`public static double median(double[] x)`	медиана
66	`public static double skewness(double[] x)`	асимметрия

83	`public static double kurtosisExcess(double[] x)`	эксцесс (у нормального = 0)
100	`public static double[] meanCI(double[] x)`	95% интервал среднего по Стьюденту
109	`public static double[] wilson(int k, int n)`	95% интервал Уилсона для доли
119	`public static double jarqueBeraP(double[] x)`	тест нормальности Жарка–Бера
128	`public static double[] ranks(double[] x)`	ранги со средним рангом для одинаковых значений
146	`public static double pearson(double[] a, double[] b)`	корреляция Пирсона
165	`public static double[][] bothPresent(double[] a, double[] b)`	пары, где заполнены обе колонки
179	`public static List<double[]> histogram(double[] x)`	столбики гистограммы по правилу Фридмана–Диакониса
218	`public static double niceStep(double v)`	округление шага до 1, 2, 2,5, 5, 10 × 10^n
237	`public static String stars(double p)`	звёздочки значимости *, **, ***

6.1.13 FinalProbabilityDistributions `mathematics/FinalProbabilityDistributions.java · 41 строк`

Final-класс

```
public final class FinalProbabilityDistributions
```

p-значения по распределениям Стьюдента, χ^2, Фишера и нормальному (обёртка над Apache Commons Math).

> **Почему такой вид.** Final-утилита. Константа NORMAL — private static final: одно стандартное нормальное распределение на всю программу.

Стр.	Объявление	Что делает
12	`private FinalProbabilityDistributions()`	конструктор
15	`public static double tP(double t, double df)`	двусторонний p по t
21	`public static double tQuantile(double df, double probability)`	квантиль Стьюдента
25	`public static double chi2P(double x, double df)`	p по χ^2
31	`public static double fP(double f, double df1, double df2)`	p по F
37	`public static double zP(double z)`	двусторонний p по z

6.1.14 FinalMultipleComparisons `mathematics/FinalMultipleComparisons.java · 34 строк`

Final-класс

```
public final class FinalMultipleComparisons
```

Поправка Бенджамини–Хохберга: из списка p-значений получает q-значения.

> **Почему такой вид.** Final-утилита. applyFdr принимает List<? extends InterfaceSignificanceTest> — список любых объектов, реализующих интерфейс.

mathematics/FinalMultipleComparisons.java — файл целиком

```java
package com.analytics.brain.mathematics;

import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

public final class FinalMultipleComparisons {

    private FinalMultipleComparisons() {
    }

    public static double[] benjaminiHochberg(double[] p) {
        int m = p.length;
        Integer[] order = new Integer[m];
        for (int i = 0; i < m; i++) order[i] = i;
        Arrays.sort(order, Comparator.comparingDouble(i -> p[i]));

        double[] q = new double[m];
        double prev = 1;
        for (int rank = m - 1; rank >= 0; rank--) {
            int i = order[rank];
            prev = Math.min(prev, p[i] * m / (rank + 1));
            q[i] = Math.min(1, prev);
        }
        return q;
    }

    public static void applyFdr(List<? extends InterfaceSignificanceTest> tests) {
        double[] p = new double[tests.size()];
        for (int i = 0; i < p.length; i++) p[i] = tests.get(i).pValue();
        double[] q = benjaminiHochberg(p);
        for (int i = 0; i < q.length; i++) tests.get(i).acceptQValue(q[i]);
    }
}
```

6.2. Пакет analysis — анализы

6.2.1 AbstractAnalitikAnalysis `analysis/AbstractAnalitikAnalysis.java · 38 строк`

Абстрактный класс наследники: `AbstractAnomalyAnalysis`, `FinalAbcAnalysis`, `FinalDriversAnalysis`, `FinalFinanceAnalysis`, `FinalQualityAnalysis`, `FinalTrendAnalysis`

```
public abstract class AbstractAnalitikAnalysis
```

База шести анализов. Хранит набор данных и задаёт общий порядок: создать таблицу → посчитать → подпись = итог → одна ключевая колонка.

> **Почему такой вид.** Абстрактный класс с шаблонным методом (паттерн Template Method): run() — final, compute() — abstract.

analysis/AbstractAnalitikAnalysis.java — файл целиком

```java
package com.analytics.brain.analysis;

import com.analytics.brain.data.FinalDataset;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public abstract class AbstractAnalitikAnalysis {

    protected final FinalDataset ds;

    protected AbstractAnalitikAnalysis(FinalDataset ds) {
        this.ds = ds;
    }

    public final Map<String, Object> run() {
        Map<String, Object> result = new LinkedHashMap<>();
        result.put("type", "table");
        compute(result);
        result.put("subtitle", result.get("summary"));
        result.put("keyCols", 1);
        return result;
    }

    protected abstract void compute(Map<String, Object> result);

    protected static Map<String, Object> series(String name, List<?> data) {
        Map<String, Object> s = new LinkedHashMap<>();
        s.put("name", name);
        s.put("data", data);
        return s;
    }

    protected static <T> List<T> first(List<T> list, int count) {
        return list.size() > count ? list.subList(0, count) : list;
    }
}
```

6.2.2 FinalDriversAnalysis `analysis/FinalDriversAnalysis.java · 272 строк`

Final-класс наследует `AbstractAnalitikAnalysis`

```
public final class FinalDriversAnalysis extends AbstractAnalitikAnalysis
```

Что влияет на метрику при прочих равных: множественная регрессия МНК с отбором факторов.

> **Почему такой вид.** Final-наследник. Внутри — статический вложенный класс Factor (один столбец матрицы признаков).

Стр.	Объявление	Что делает
40	`public FinalDriversAnalysis(FinalDataset ds, String target, List<String> factorNames, List<FinalCo…`	конструктор
65	`protected void compute(Map<String, Object> result)`	цель → кандидаты → без утечек → полные строки → матрица → без коллинеарных → МНК → t-тесты → поправка → таблица
158	`private List<FinalColumn> withoutLeaks(FinalColumn y, List<FinalColumn> candidates, List<Str…`	убирает колонки, посчитанные из цели, и почти равные ей (связь ≥ 0,9)
181	`private static List<Integer> completeRows(int[] idx, FinalColumn y, List<FinalColumn> used)`	строки, где заполнены цель и все факторы
198	`private static List<Factor> designMatrix(List<FinalColumn> used, List<Integer> rows, Map&…`	z-оценки для чисел, дамми 0/1 для категорий
233	`private static List<Factor> dropCollinear(List<Factor> factors, double[] yv, int n, List<Str…`	жадный отбор с проверкой QR-разложением, не больше n/15 факторов
266	`private static boolean isBinary(double[] v)`	цель из одних 0 и 1?

6.2.3 FinalAbcAnalysis `analysis/FinalAbcAnalysis.java · 91 строк`

Final-класс наследует `AbstractAnalitikAnalysis`

```
public final class FinalAbcAnalysis extends AbstractAnalitikAnalysis
```

ABC-анализ: какие группы дают 80% результата (A), следующие 15% (B) и хвост (C). Возвращает таблицу и график Парето.

> **Почему такой вид.** Final-наследник, только compute().

Стр.	Объявление	Что делает
26	`public FinalAbcAnalysis(FinalDataset ds, String dim, String value, List<FinalCondition> filters)`	конструктор
34	`protected void compute(Map<String, Object> result)`	суммы по группам → сортировка → накопленная доля → класс

6.2.4 FinalTrendAnalysis `analysis/FinalTrendAnalysis.java · 194 строк`

Final-класс наследует `AbstractAnalitikAnalysis`

```
public final class FinalTrendAnalysis extends AbstractAnalitikAnalysis
```

Динамика по периодам: изменение к прошлому периоду, скользящее среднее, линейный тренд с проверкой наклона и прогноз с 95% интервалом предсказания.

> **Почему такой вид.** Final-наследник.

Стр.	Объявление	Что делает
37	`public FinalTrendAnalysis(FinalDataset ds, String metric, String agg, String dateCol, String granularity,`	конструктор
49	`protected void compute(Map<String, Object> result)`	периоды → агрегаты → МНК по номеру периода → t-тест наклона → прогноз
164	`private static long periodKey(LocalDate d, String step)`	ключ периода (день, неделя, месяц, квартал, год)
179	`private static String periodLabel(LocalDate d, String step)`	подпись периода

6.2.5 FinalFinanceAnalysis `analysis/FinalFinanceAnalysis.java · 179 строк`

Final-класс наследует `AbstractAnalitikAnalysis`

```
public final class FinalFinanceAnalysis extends AbstractAnalitikAnalysis
```

Финансовая сводка: выручка, доля платящих, ARPU, ARPPU, средний чек, маржа, оценка скидок, концентрация выручки (топ-20% и Джини).

> **Почему такой вид.** Final-наследник.

Стр.	Объявление	Что делает
30	`public FinalFinanceAnalysis(FinalDataset ds, String revenue, String cost, String orders, String groupBy,`	конструктор
41	`protected void compute(Map<String, Object> result)`	находит колонки → строки по сегментам и «Итого» → концентрация
123	`private static double gini(double[] sorted, double total)`	коэффициент Джини по отсортированной выручке
131	`private FinalColumn findColumn(String given, String... words)`	колонка по имени или по словам «выручка», «cost»…
141	`private static List<Object> financeRow(String name, List<Integer> rows, FinalColumn rev, FinalC…`	одна строка финансовой таблицы

6.2.6 FinalQualityAnalysis `analysis/FinalQualityAnalysis.java · 83 строк`

Final-класс наследует `AbstractAnalitikAnalysis`

```
public final class FinalQualityAnalysis extends AbstractAnalitikAnalysis
```

Аудит качества данных: пропуски, дубликаты, константы, экстремумы, асимметрия, «Другое».

> **Почему такой вид.** Final-наследник.

Стр.	Объявление	Что делает
22	`public FinalQualityAnalysis(FinalDataset ds)`	конструктор
27	`protected void compute(Map<String, Object> result)`	дубликаты строк и замечания по каждой колонке
53	`private List<String> notes(FinalColumn c)`	список замечаний по одной колонке

6.2.7 AbstractAnomalyAnalysis `analysis/AbstractAnomalyAnalysis.java · 38 строк`

Абстрактный класс наследует `AbstractAnalitikAnalysis` · наследники: `FinalGroupAnomalyAnalysis`, `FinalRecordAnomalyAnalysis`

```
public abstract class AbstractAnomalyAnalysis extends AbstractAnalitikAnalysis
```

Общая часть двух способов искать аномалии. compute() здесь final: найти метрику, отфильтровать строки, вызвать findAnomalies, положить строки в результат.

> **Почему такой вид.** Абстрактный класс в середине иерархии: сам наследник AbstractAnalitikAnalysis и сам родитель двух final-классов. Статический метод create — фабрика, которая по параметру by выбирает нужного наследника.

analysis/AbstractAnomalyAnalysis.java — файл целиком

```
1  package com.analytics.brain.analysis;
2
3  import com.analytics.brain.data.FinalColumn;
4  import com.analytics.brain.data.FinalDataset;
5  import com.analytics.brain.query.FinalCondition;
6  import com.analytics.brain.query.FinalQueryEngine;
7
8  import java.util.ArrayList;
9  import java.util.List;
10 import java.util.Map;
11
12 public abstract class AbstractAnomalyAnalysis extends AbstractAnalitikAnalysis {
13
14     protected final String metric;
15     protected final List<FinalCondition> filters;
16
17     protected AbstractAnomalyAnalysis(FinalDataset ds, String metric, List<FinalCondition>
   filters) {
18         super(ds);
19         this.metric = metric;
20         this.filters = filters;
21     }
22
23     public static AbstractAnomalyAnalysis create(FinalDataset ds, String metric, String by,
   List<FinalCondition> filters) {
24         if (by != null && !by.isBlank()) return new FinalGroupAnomalyAnalysis(ds, metric, by,
   filters);
25         return new FinalRecordAnomalyAnalysis(ds, metric, filters);
26     }
27
28     @Override
29     protected final void compute(Map<String, Object> result) {
30         FinalColumn m = FinalQueryEngine.need(ds, metric);
31         int[] idx = FinalQueryEngine.filter(ds, filters);
32         List<List<Object>> rows = new ArrayList<>();
33         findAnomalies(m, idx, rows, result);
34         result.put("rows", rows);
35     }
36
37     protected abstract void findAnomalies(FinalColumn m, int[] idx, List<List<Object>> rows,
   Map<String, Object> result);
38 }
```

6.2.8 FinalGroupAnomalyAnalysis `analysis/FinalGroupAnomalyAnalysis.java · 64 строк`

Final-класс наследует `AbstractAnomalyAnalysis`

```
public final class FinalGroupAnomalyAnalysis extends AbstractAnomalyAnalysis
```

Аномальные группы: среднее каждой группы сравнивается с медианой средних через робастную z-оценку (MAD).

> **Почему такой вид.** Final-наследник AbstractAnomalyAnalysis.

Стр.	Объявление	Что делает
23	`public FinalGroupAnomalyAnalysis(FinalDataset ds, String metric, String by, List<FinalCondition> filt…`	конструктор
29	`protected void findAnomalies(FinalColumn m, int[] idx, List<List<Object>> rows, Map<String, …`	средние групп → медиана → MAD → z → статус

6.2.9 FinalRecordAnomalyAnalysis `analysis/FinalRecordAnomalyAnalysis.java · 58 строк`

Final-класс наследует `AbstractAnomalyAnalysis`

```
public final class FinalRecordAnomalyAnalysis extends AbstractAnomalyAnalysis
```

Экстремальные записи по правилу Тьюки с порогом 3×IQR.

> **Почему такой вид.** Final-наследник AbstractAnomalyAnalysis.

Стр.	Объявление	Что делает
23	`public FinalRecordAnomalyAnalysis(FinalDataset ds, String metric, List<FinalCondition> filters)`	конструктор
28	`protected void findAnomalies(FinalColumn m, int[] idx, List<List<Object>> rows, Map<String, …`	квартили → границы → записи за границами

6.3. Пакет formula — интерпретатор формул

6.3.1 InterfaceFormulaNode `formula/InterfaceFormulaNode.java · 6 строк`

Интерфейс наследники: `FinalBinaryNode`, `FinalCellNode`, `FinalConstantNode`, `FinalFunctionNode`, `FinalUnaryNode`

```
public interface InterfaceFormulaNode
```

Узел дерева формулы. Один метод: вычислить значение для строки row.

> **Почему такой вид.** Интерфейс: у узлов нет общих данных, только общее умение. Это классический паттерн «Интерпретатор».

formula/InterfaceFormulaNode.java — файл целиком

```
1  package com.analytics.brain.formula;
2
3  public interface InterfaceFormulaNode {
4
5      Object evaluate(int row);
6  }
```

6.3.2 FinalConstantNode `formula/FinalConstantNode.java · 15 строк`

Final-класс реализует `InterfaceFormulaNode`

```
public final class FinalConstantNode implements InterfaceFormulaNode
```

Лист дерева: число, строка или true/false из текста формулы.

> **Почему такой вид.** Final, неизменяемый (private final value).

formula/FinalConstantNode.java — файл целиком

```
1  package com.analytics.brain.formula;
2
3  public final class FinalConstantNode implements InterfaceFormulaNode {
4
5      private final Object value;
6
7      public FinalConstantNode(Object value) {
8          this.value = value;
9      }
10
11     @Override
12     public Object evaluate(int row) {
13         return value;
14     }
15 }
```

6.3.3 FinalCellNode `formula/FinalCellNode.java · 38 строк`

Final-класс реализует `InterfaceFormulaNode`

```
public final class FinalCellNode implements InterfaceFormulaNode
```

Лист дерева: ссылка на колонку [Выручка]. Номер колонки ищется один раз при разборе.

> **Почему такой вид.** Final. Бросает ExceptionAnalitikData, если колонки нет.

Стр.	Объявление	Что делает
15	`public FinalCellNode(FinalDataset ds, String name)`	ищет колонку (без учёта регистра)
29	`public Object evaluate(int row)`	значение ячейки: число, если колонка числовая, иначе текст

6.3.4 FinalUnaryNode `formula/FinalUnaryNode.java · 17 строк`

Final-класс реализует `InterfaceFormulaNode`

```
public final class FinalUnaryNode implements InterfaceFormulaNode
```

Операция с одним операндом: −x или not x.

> **Почему такой вид.** Final; сама операция — константа EnumFormulaOperator.

formula/FinalUnaryNode.java — файл целиком

```
1  package com.analytics.brain.formula;
2
3  public final class FinalUnaryNode implements InterfaceFormulaNode {
4
5      private final EnumFormulaOperator operator;
6      private final InterfaceFormulaNode operand;
7
8      public FinalUnaryNode(EnumFormulaOperator operator, InterfaceFormulaNode operand) {
9          this.operator = operator;
10         this.operand = operand;
11     }
12
13     @Override
14     public Object evaluate(int row) {
15         return operator.apply(operand.evaluate(row), null);
16     }
17 }
```

6.3.5 FinalBinaryNode `formula/FinalBinaryNode.java · 21 строк`

Final-класс реализует `InterfaceFormulaNode`

```
public final class FinalBinaryNode implements InterfaceFormulaNode
```

Операция с двумя операндами: +, −, *, /, %, сравнения, and, or.

> **Почему такой вид.** Final; поля left/right имеют тип интерфейса — туда подходит любой узел.

Стр.	Объявление	Что делает
9	`public FinalBinaryNode(EnumFormulaOperator operator, InterfaceFormulaNode left, InterfaceFormulaNode right)`	конструктор
16	`public Object evaluate(int row)`	вычислить левое, правое, применить операцию

6.3.6 FinalFunctionNode `formula/FinalFunctionNode.java · 22 строк`

Final-класс реализует `InterfaceFormulaNode`

```
public final class FinalFunctionNode implements InterfaceFormulaNode
```

Вызов функции: if(…), round(…), bucket(…) и т. д.

> **Почему такой вид.** Final; функция — константа EnumFormulaFunction.

Стр.	Объявление	Что делает
11	`public FinalFunctionNode(EnumFormulaFunction function, List<InterfaceFormulaNode> arguments)`	конструктор
17	`public Object evaluate(int row)`	вычислить все аргументы и вызвать функцию

6.3.7 EnumFormulaOperator `formula/EnumFormulaOperator.java · 111 строк`

Перечисление

```
public enum EnumFormulaOperator
```

15 операций формулы. У каждой константы своё тело метода apply.

> **Почему такой вид.** Enum с абстрактным методом: каждая константа — анонимный подкласс перечисления. Это полиморфизм без отдельных файлов.

Константы: `OR`, `AND`, `GREATER_OR_EQUAL`, `LESS_OR_EQUAL`, `NOT_EQUAL`, `EQUAL`, `GREATER`, `LESS`, `PLUS`, `MINUS`, `MULTIPLY`, `DIVIDE`, `MODULO`, `NEGATE`, `NOT`

Стр.	Объявление	Что делает
102	`public abstract Object apply(Object a, Object b);`	abstract: применить операцию
104	`private static boolean bothNumbers(Object a, Object b)`	оба операнда — числа?
108	`private static boolean sameText(Object a, Object b)`	строки равны без учёта регистра?

6.3.8 EnumFormulaFunction `formula/EnumFormulaFunction.java · 142 строк`

Перечисление

```
public enum EnumFormulaFunction
```

15 функций формулы с русскими синонимами (если, округл).

> **Почему такой вид.** Enum с полями (список имён), конструктором и абстрактным методом call.

Константы: `IF`, `ABS`, `ROUND`, `LOG`, `SQRT`, `MIN`, `MAX`, `COALESCE`, `LOWER`, `UPPER`, `CONTAINS`, `YEAR`, `MONTH`, `WEEKDAY`, `BUCKET`

Стр.	Объявление	Что делает
123	`EnumFormulaFunction(String... names)`	конструктор: имена функции
127	`public abstract Object call(List<Object> args);`	abstract: вычислить функцию

129	`public static EnumFormulaFunction byName(String name)`	static: найти функцию по имени или бросить ошибку
139	`private static LocalDate date(List<Object> args)`	private static: разобрать дату из аргумента

6.3.9 FinalFormulaParser `formula/FinalFormulaParser.java · 183 строк`

Final-класс

`public final class FinalFormulaParser`

Парсер рекурсивным спуском: превращает текст формулы в дерево узлов. Каждому уровню приоритета — свой метод: or → and → сравнение → + − → * / % → унарные → атом.

> **Почему такой вид.** Final, закрытый конструктор, снаружи только static parse(). Поле pos — где сейчас читаем текст.

Стр.	Объявление	Что делает
24	`private FinalFormulaParser(String src, FinalDataset ds)`	конструктор
29	`public static InterfaceFormulaNode parse(String src, FinalDataset ds)`	static: разобрать всю формулу и проверить, что текст кончился
39	`private InterfaceFormulaNode or()`	уровень ||, or, или
47	`private InterfaceFormulaNode and()`	уровень &&, and, и
55	`private InterfaceFormulaNode comparison()`	одно сравнение (двухсимвольные операторы проверяются раньше односимвольных)
63	`private InterfaceFormulaNode sum()`	+ и −
76	`private InterfaceFormulaNode product()`	*, / и %
91	`private InterfaceFormulaNode unary()`	унарный минус и not
97	`private InterfaceFormulaNode atom()`	скобки, [колонка], строка, число, слово
134	`private InterfaceFormulaNode wordOrFunction()`	true/false, имя колонки или вызов функции
159	`private void skipSpaces()`	пропустить пробелы
163	`private boolean eat(String token)`	съесть токен, если он дальше
172	`private boolean eatWord(String word)`	съесть слово целиком («и» не съест начало «имя»)

6.3.10 FinalFormulaValues `formula/FinalFormulaValues.java · 29 строк`

Final-класс

`public final class FinalFormulaValues`

Приведение значений формулы к числу, логике и тексту.

> **Почему такой вид.** Final-утилита со static-методами.

Стр.	Объявление	Что делает
8	`private FinalFormulaValues()`	конструктор
11	`public static double number(Object o)`	Double / Boolean / строка → число

18	`public static boolean bool(Object o)`	значение → true/false
24	`public static String text(Object o)`	значение → строка

6.4. Пакет data — данные

6.4.1 EnumColumnType `data/EnumColumnType.java · 32 строк`

Перечисление

`public enum EnumColumnType`

Пять типов колонки: число, категория, дата, идентификатор, текст.

> **Почему такой вид.** Enum с полями: у каждого типа русское название и два признака.

Константы: `NUMERIC`, `CATEGORICAL`, `DATE`, `ID`, `TEXT`

Стр.	Объявление	Что делает
15	`EnumColumnType(String russianName, boolean analyzable, boolean dimension)`	конструктор перечисления
21	`public String russianName()`	русское название
25	`public boolean analyzable()`	участвует в анализе?
29	`public boolean dimension()`	можно ли резать выборку?

6.4.2 FinalColumn `data/FinalColumn.java · 32 строк`

Final-класс

`public final class FinalColumn`

Одна колонка после определения типа: числа (num), категории (cat), уровни, пропуски, уникальные значения, признак «счётчика».

> **Почему такой вид.** Final. Поле name — final (имя колонки не меняется). Поведение делегируется перечислению типа.

data/FinalColumn.java — файл целиком

```java
package com.analytics.brain.data;

import java.util.List;

public final class FinalColumn {

    public final String name;
    public EnumColumnType type;
    public double[] num;
    public String[] cat;
    public List<String> levels;
    public int missing;
    public int unique;
    public String granularity;
    public boolean countLike;

    public FinalColumn(String name) {
        this.name = name;
    }

    public boolean analyzable() {
        return type != null && type.analyzable();
    }

    public boolean isDim() {
        return type != null && type.dimension();
    }

    public String typeRu() {
        return type == null ? EnumColumnType.TEXT.russianName() : type.russianName();
    }
}
```

6.4.3 FinalDataset `data/FinalDataset.java · 68 строк`

Final-класс

`public final class FinalDataset`

Загруженная таблица: исходные строки, колонки, KPI, воронка, версия и кэш обзора.

> **Почему такой вид.** Final-класс данных. Поля overviewCache и overviewVersion помечены transient — не сохраняются при сериализации.

Стр.	Объявление	Что делает
38	`public FinalColumn col(String name)`	колонка по имени
42	`public Map<String, Object> summary()`	короткое описание набора для браузера

6.4.4 InterfaceDataParser `data/InterfaceDataParser.java · 6 строк`

Интерфейс наследники: `FinalCsvParser`, `FinalJsonParser`

`public interface InterfaceDataParser`

Контракт парсера: разобрать текст в набор данных.

> **Почему такой вид.** Интерфейс: загрузчику всё равно, CSV это или JSON.

data/InterfaceDataParser.java — файл целиком

```java
package com.analytics.brain.data;

public interface InterfaceDataParser {

    void parse(FinalDataset ds, String text) throws Exception;
}
```

6.4.5 FinalCsvParser `data/FinalCsvParser.java · 69 строк`

Final-класс реализует `InterfaceDataParser`

```
public final class FinalCsvParser implements InterfaceDataParser
```

Разбор CSV: угадывает разделитель, читает заголовок и строки (Apache Commons CSV).

> **Почему такой вид.** Final, реализует InterfaceDataParser.

Стр.	Объявление	Что делает
13	`public void parse(FinalDataset ds, String text) throws Exception`	@Override: разобрать CSV
45	`private static char guessDelimiter(String headerLine)`	разделитель — символ, которого больше всего в заголовке вне кавычек
57	`private static int countOutsideQuotes(String line, char symbol)`	подсчёт символа вне кавычек

6.4.6 FinalJsonParser `data/FinalJsonParser.java · 96 строк`

Final-класс реализует `InterfaceDataParser`

```
public final class FinalJsonParser implements InterfaceDataParser
```

Разбор JSON: находит массив записей в ответе API и разворачивает вложенные объекты в колонки user.city.

> **Почему такой вид.** Final, реализует InterfaceDataParser.

Стр.	Объявление	Что делает
20	`public void parse(FinalDataset ds, String text) throws Exception`	@Override: разобрать JSON
29	`public static Map<String, String> flatObject(JsonNode node)`	static: развернуть вложенный объект
34	`private static void flatten(String prefix, JsonNode node, Map<String, String> out)`	рекурсивный обход
57	`private static JsonNode findArray(JsonNode node, int depth)`	поиск массива записей (не глубже 4 уровней)
83	`private static JsonNode arrayOfArrays(JsonNode rows)`	[[заголовки], [значения]...] → объекты

6.4.7 FinalDataLoader `data/FinalDataLoader.java · 437 строк`

Final-класс

`public final class FinalDataLoader`

Загрузка по ссылке, выбор парсера, определение типов колонок, поиск KPI, воронки и «сущности» (пользователей, заказов…).

> **Почему такой вид.** Final-утилита. Самые важные константы — static final: слова KPI, слова этапов воронки, форматы дат, значения-пропуски.

Стр.	Объявление	Что делает
34	`private FinalDataLoader()`	конструктор
74	`public static String[] fetch(String url, Map<String, String> headers) throws Exception`	скачать данные по ссылке (Google-таблица → CSV)
102	`public static String googleSheetsToCsv(String url)`	ссылка на Google-таблицу → ссылка на выгрузку CSV
113	`public static void parseAuto(FinalDataset ds, String body, String contentType) throws Exception`	JSON или CSV — выбрать парсер и вызвать через интерфейс
123	`public static void setRows(FinalDataset ds, List<Map<String, String>> rows)`	заголовок = объединение ключей всех строк
131	`public static String[] toRecord(List<String> header, Map<String, String> row)`	строка-словарь → массив значений
140	`public static void build(FinalDataset ds)`	собрать колонки из строк
163	`public static FinalColumn infer(String name, String[] raw, int n)`	определить тип колонки (порядок правил важен)
217	`private static void makeNumeric(FinalColumn column, String name, String[] raw, double[] num,`	числовая колонка или ID; признак «счётчика»
245	`private static void makeCategorical(FinalColumn column, String[] raw, int n)`	14 самых частых значений, остальное — «Другое»
279	`private static void makeDate(FinalColumn column, long[] days, int n)`	дата: номер дня и период
339	`public static double parseNum(String s)`	число из строки («1 250,5», «15%»)
356	`public static LocalDate parseDate(String s)`	дата в одном из форматов
369	`public static void detectKpis(FinalDataset ds)`	KPI по словам в названиях
388	`public static void detectFunnel(FinalDataset ds)`	колонка этапов воронки
408	`private static int stagePosition(String stage)`	порядок этапа
416	`public static void detectEntity(FinalDataset ds)`	что считаем: пользователей, заказы…

6.4.8 FinalRows `data/FinalRows.java · 54 строк`

Final-класс

`public final class FinalRows`

Работа с выборками строк. Сегмент в проекте — просто массив номеров строк int[].

> **Почему такой вид.** Final-утилита.

Стр.	Объявление	Что делает
10	`private FinalRows()`	конструктор

13	`public static int[] all(FinalDataset ds)`	все строки
19	`public static int[] matching(FinalDataset ds, List<FinalGraphFilter> path)`	строки под путь фильтров
29	`public static int[] where(int[] rows, FinalColumn c, String value)`	строки со значением
33	`public static int[] whereNot(int[] rows, FinalColumn c, String value)`	строки с другим значением
37	`public static int[] restOf(FinalDataset ds, List<FinalGraphFilter> path)`	«остальные внутри родителя» для сегмента
43	`public static double[] values(FinalColumn c, int[] rows)`	числа колонки для строк
49	`public static String[] labels(FinalColumn c, int[] rows)`	категории колонки для строк

6.4.9 FinalDemoData `data/FinalDemoData.java · 148 строк`

Final-класс

```
public final class FinalDemoData
```

Генератор демо-набора «пользователи сайта» с заложенными зависимостями.

> **Почему такой вид.** Final-утилита; фиксированное зерно генератора — одинаковые данные при каждом запуске.

Стр.	Объявление	Что делает
14	`private FinalDemoData()`	конструктор
17	`private static double rnd()`	следующее случайное число
22	`private static <T> T pick(T[] options)`	случайный элемент
26	`private static double clamp(double v, double lo, double hi)`	ограничить значение
30	`private static double round2(double v)`	округлить до сотых
34	`public static synchronized List<Map<String, Object>> users(int count, boolean live)`	сгенерировать N пользователей
133	`public static String toCsv(List<Map<String, Object>> rows)`	записать в CSV

6.5. Пакет query — сводные и графики

6.5.1 FinalChartSpec `query/FinalChartSpec.java · 16 строк`

Final-класс

`public final class FinalChartSpec`

Описание графика: тип, оси x и y, агрегат, группировка, фильтры.

> **Почему такой вид.** Final-класс данных (DTO): только публичные поля, Jackson заполняет их из JSON запроса.

query/FinalChartSpec.java — файл целиком

```
1  package com.analytics.brain.query;
2
3  import java.util.ArrayList;
4  import java.util.List;
5
6  public final class FinalChartSpec {
7
8      public String type = "bar";
9      public String x;
10     public String y;
11     public String agg;
12     public String group;
13     public String title;
14     public List<FinalCondition> filters = new ArrayList<>();
15     public Integer top;
16 }
```

6.5.2 FinalCondition `query/FinalCondition.java · 11 строк`

Final-класс

`public final class FinalCondition`

Условие фильтра: колонка, операция (=, !=, in, >, contains…), значение или список значений.

> **Почему такой вид.** Final-класс данных (DTO): только публичные поля, Jackson заполняет их из JSON запроса.

query/FinalCondition.java — файл целиком

```
1  package com.analytics.brain.query;
2
3  import java.util.List;
4
5  public final class FinalCondition {
6
7      public String column;
8      public String op = "=";
9      public String value;
10     public List<String> values;
11 }
```

6.5.3 FinalGrouping `query/FinalGrouping.java · 9 строк`

Final-класс

`public final class FinalGrouping`

Результат группировки: метка группы для каждой строки и порядок групп.

> **Почему такой вид.** Final-класс данных (DTO): только публичные поля, Jackson заполняет их из JSON запроса.

query/FinalGrouping.java — файл целиком

```java
package com.analytics.brain.query;

import java.util.List;

public final class FinalGrouping {

    public String[] labels;
    public List<String> order;
}
```

6.5.4 FinalPivotSpec `query/FinalPivotSpec.java · 16 строк`

Final-класс

`public final class FinalPivotSpec`

Описание сводной таблицы: строки, столбцы, значения, агрегат, фильтры, сортировка, топ.

> **Почему такой вид.** Final-класс данных (DTO): только публичные поля, Jackson заполняет их из JSON запроса.

query/FinalPivotSpec.java — файл целиком

```java
package com.analytics.brain.query;

import java.util.ArrayList;
import java.util.List;

public final class FinalPivotSpec {

    public List<String> rows = new ArrayList<>();
    public String columns;
    public String values;
    public String agg = "count";
    public String title;
    public String sort;
    public List<FinalCondition> filters = new ArrayList<>();
    public Integer top;
}
```

6.5.5 FinalQueryEngine `query/FinalQueryEngine.java · 848 строк`

Final-класс

`public final class FinalQueryEngine`

Сводные таблицы, графики, фильтры, сравнение групп, воронка, новая колонка и срез данных — вкладка «Сводные» и половина инструментов ИИ.

> **Почему такой вид.** Final-утилита. Спецификации запросов вынесены в отдельные классы данных FinalPivotSpec, FinalChartSpec, FinalCondition, FinalGrouping.

Стр.	Объявление	Что делает

58	`private FinalQueryEngine()`	конструктор
61	`public static int[] filter(FinalDataset ds, List<FinalCondition> conds)`	номера строк под все условия
77	`public static boolean matches(FinalDataset ds, FinalColumn col, int j, int i, FinalCondition cond)`	подходит ли строка под одно условие
141	`public static String describe(List<FinalCondition> conds)`	условия словами
149	`public static FinalGrouping groups(FinalDataset ds, FinalColumn c, int[] rows)`	как разбить строки по колонке
155	`private static FinalGrouping groupByCategory(FinalDataset ds, FinalColumn c, int[] rows)`	по категориям
171	`private static FinalGrouping groupByNumber(FinalDataset ds, FinalColumn c, int[] rows)`	числа: по значению или квартилям
210	`private static FinalGrouping groupByText(FinalDataset ds, FinalColumn c, int[] rows)`	по тексту
227	`public static double agg(FinalDataset ds, FinalColumn v, List<Integer> rows, String agg, int total)`	агрегат: count, share, sum, mean, median, min, max, rate, distinct
273	`public static String aggFmt(String agg, FinalColumn v)`	формат числа агрегата
279	`public static Map<String, Object> pivot(FinalDataset ds, FinalPivotSpec spec)`	сводная таблица
419	`private static Map<List<String>, List<Integer>> bucketRows(int[] idx, List<FinalGroupi…`	строки по ячейкам сводной
458	`public static Map<String, Object> chart(FinalDataset ds, FinalChartSpec spec)`	данные для графика
479	`private static void histogramChart(FinalDataset ds, FinalChartSpec spec, int[] idx, Map<String, Object&g…`	гистограмма
496	`private static void scatterChart(FinalDataset ds, FinalChartSpec spec, int[] idx, Map<String, Object>…`	точечный график
514	`private static void categoryChart(FinalDataset ds, FinalChartSpec spec, int[] idx, Map<String, Object>…`	столбцы/линии/круг
572	`private static Map<String, Object> series(String name, List<Object> data)`	серия графика
579	`public static Map<String, Object> compare(FinalDataset ds, String metric, String groupBy, List<Fin…`	сравнение групп с тестом
613	`private static String compareNumeric(FinalColumn m, FinalGrouping groups, int[] idx, List<List<Object…`	числа: ANOVA + Уэлч для каждой группы
647	`private static String compareCategories(FinalDataset ds, FinalColumn m, FinalGrouping groups, int[] idx,`	категории: χ^2
682	`public static Map<String, Object> funnel(FinalDataset ds, String by, List<FinalCondition> filters)`	воронка по этапам

737	`public static Map<String, Object> addColumn(FinalDataset ds, String name, String formula)`	вычисляемая колонка по формуле
794	`private static String cellText(Object v)`	значение формулы → текст ячейки
804	`public static FinalDataset subset(FinalDataset ds, String name, List<FinalCondition> filters)`	срез набора по фильтрам
817	`public static FinalColumn need(FinalDataset ds, String name)`	колонка или ошибка «Нет колонки»
830	`public static double[] valuesWhere(FinalColumn c, int[] idx, IntPredicate condition)`	значения по условию
834	`public static Double num(double v)`	округление для JSON
839	`public static double asNumber(Object o)`	объект → число
843	`public static String trimNum(double v)`	число без лишних нулей

6.6. Пакет graph — объекты графа

6.6.1 FinalGraphBar `graph/FinalGraphBar.java · 22 строк`

Final-класс

```
public final class FinalGraphBar
```

Столбик мини-диаграммы в карточке.

> **Почему такой вид.** Final-класс данных: объект создаётся, заполняется и целиком превращается в JSON.

Стр.	Объявление	Что делает
12	`public FinalGraphBar()`	конструктор (перегружен: пустой и с параметрами)
15	`public FinalGraphBar(String label, double value, double lo, double hi, String text)`	конструктор (перегружен: пустой и с параметрами)

6.6.2 FinalGraphEffect `graph/FinalGraphEffect.java · 12 строк`

Final-класс

```
public final class FinalGraphEffect
```

Связь «сегмент влияет на метрику».

> **Почему такой вид.** Final-класс данных: объект создаётся, заполняется и целиком превращается в JSON.

6.6.3 FinalGraphFilter `graph/FinalGraphFilter.java · 19 строк`

Final-класс

```
public final class FinalGraphFilter
```

Условие пути «колонка = значение». В JSON поля называются c и v (аннотация @JsonProperty).

> **Почему такой вид.** Final-класс данных: объект создаётся, заполняется и целиком превращается в JSON.

Стр.	Объявление	Что делает
12	`public FinalGraphFilter()`	конструктор (перегружен)
15	`public FinalGraphFilter(String column, String value)`	конструктор (перегружен)

6.6.4 FinalGraphInsight `graph/FinalGraphInsight.java · 13 строк`

Final-класс

```
public final class FinalGraphInsight
```

«Ключевая находка» для левой панели.

> **Почему такой вид.** Final-класс данных: объект создаётся, заполняется и целиком превращается в JSON.

6.6.5 FinalGraphLink `graph/FinalGraphLink.java · 26 строк`

Final-класс

`public final class FinalGraphLink`

Связь графа. Поле test имеет тип AbstractAnalitikMathematicsTest — туда кладётся любой конкретный тест.

> **Почему такой вид.** Final-класс данных: объект создаётся, заполняется и целиком превращается в JSON.

graph/FinalGraphLink.java — файл целиком

```
1  package com.analytics.brain.graph;
2
3  import com.analytics.brain.mathematics.AbstractAnalitikMathematicsTest;
4
5  public final class FinalGraphLink {
6
7      public String source;
8      public String target;
9      public String type;
10     public String label;
11     public String level = "none";
12     public String direction;
13     public String text;
14     public double strength;
15     public AbstractAnalitikMathematicsTest test;
16
17     public FinalGraphLink() {
18     }
19
20     public FinalGraphLink(String source, String target, String type, String label) {
21         this.source = source;
22         this.target = target;
23         this.type = type;
24         this.label = label;
25     }
26 }
```

6.6.6 FinalGraphNode `graph/FinalGraphNode.java · 32 строк`

Final-класс

`public final class FinalGraphNode`

Узел графа: вид, подписи, уровень, размер, путь, статистика для карточки.

> **Почему такой вид.** Final-класс данных: объект создаётся, заполняется и целиком превращается в JSON.

graph/FinalGraphNode.java — файл целиком

```java
package com.analytics.brain.graph;

import java.util.ArrayList;
import java.util.List;

public final class FinalGraphNode {

    public String id;
    public String kind;
    public String label;
    public String sub;
    public String parent;
    public String column;
    public String value;
    public String level = "none";
    public String direction;
    public int n;
    public Double share;
    public boolean expandable;
    public FinalGraphRef ref;
    public String text;
    public List<Object[]> stats = new ArrayList<>();
    public FinalGraphTable table;
    public String barsTitle;
    public List<FinalGraphBar> bars;
    public List<FinalGraphEffect> effects;

    public FinalGraphNode stat(String name, Object value) {
        stats.add(new Object[]{name, value});
        return this;
    }
}
```

6.6.7 FinalGraphPayload `graph/FinalGraphPayload.java · 14 строк`

Final-класс

`public final class FinalGraphPayload`

Ответ сервера на обзор и раскрытие: узлы, связи, находки.

> **Почему такой вид.** Final-класс данных: объект создаётся, заполняется и целиком превращается в JSON.

6.6.8 FinalGraphRef `graph/FinalGraphRef.java · 12 строк`

Final-класс

`public final class FinalGraphRef`

Что нужно серверу, чтобы раскрыть узел: вид узла, путь фильтров, колонка, id узла.

> **Почему такой вид.** Final-класс данных: объект создаётся, заполняется и целиком превращается в JSON.

6.6.9 FinalGraphTable `graph/FinalGraphTable.java · 25 строк`

Final-класс

`public final class FinalGraphTable`

Таблица в карточке узла. Два перегруженных конструктора и метод row(…), который возвращает this — вызовы можно писать цепочкой.

> **Почему такой вид.** Final-класс данных: объект создаётся, заполняется и целиком превращается в JSON.

Стр.	Объявление	Что делает
13	`public FinalGraphTable()`	конструктор (перегружен: пустой и с заголовком)
16	`public FinalGraphTable(String title, String... head)`	конструктор (перегружен: пустой и с заголовком)
21	`public FinalGraphTable row(Object... cells)`	добавить строку и вернуть this — вызовы можно писать цепочкой

6.7. Пакет service — построение графа

6.7.1 ServiceBrainGraph `service/ServiceBrainGraph.java · 127 строк`

Spring-сервис

```
public class ServiceBrainGraph
```

Главный сервис графа: хранит загруженные наборы и строит обзор и раскрытие веток. Сам почти не считает — раздаёт работу четырём помощникам.

> **Почему такой вид.** Spring-сервис (@Service): один объект на приложение, его получают контроллеры и ИИ. Метод overview — synchronized, чтобы два запроса не строили один кэш одновременно.

Стр.	Объявление	Что делает
30	`public synchronized FinalGraphPayload overview(FinalDataset ds)`	обзорный граф с кэшем
74	`private FinalGraphNode rootNode(FinalDataset ds)`	центр графа
101	`public FinalGraphPayload expand(FinalDataset ds, FinalGraphRef ref)`	раскрытие узла по его виду

6.7.2 FinalAssociationLinks `service/FinalAssociationLinks.java · 125 строк`

Final-класс

```
public final class FinalAssociationLinks
```

Связь двух переменных: выбирает тест по типам колонок и пишет текст подсказки.

> **Почему такой вид.** Final-утилита пакета service.

Стр.	Объявление	Что делает
18	`private FinalAssociationLinks()`	конструктор
21	`static FinalGraphLink association(FinalColumn a, FinalColumn b)`	число–число → Спирмен, категория–категория → χ^2, категория–число → ANOVA
38	`private static FinalGraphLink correlationLink(FinalColumn a, FinalColumn b, boolean aDate, boolean bDate)`	связь по Спирмену
62	`private static FinalGraphLink chiSquareLink(FinalColumn a, FinalColumn b)`	связь по χ^2, самая «перегруженная» клетка
91	`private static FinalGraphLink anovaLink(FinalColumn category, FinalColumn number)`	связь по ANOVA, группы с max и min средним

6.7.3 FinalBranchBuilder `service/FinalBranchBuilder.java · 574 строк`

Final-класс

```
public final class FinalBranchBuilder
```

Строит ветки ментальной карты: измерение → категории → метрики сегмента → воронка → вложенные измерения.

> **Почему такой вид.** Final-утилита. Внутри статический вложенный класс MetricTest. Константы TOP_SUBDIMS = 5 и MAX_KPI_LEAVES = 6 — private static final.

Стр.	Объявление	Что делает
39	`private FinalBranchBuilder()`	конструктор
57	`static void expandDimension(FinalDataset ds, List<FinalGraphFilter> path, int[] rows, FinalColumn c, …`	измерение → категории с долями
89	`private static FinalGraphLink periodFlow(FinalColumn kpi, String fromId, String toId, double before, double…`	стрелка между соседними периодами
97	`static void expandSegment(FinalDataset ds, List<FinalGraphFilter> path, int[] rows, String nodeId, Fi…`	сегмент → метрики, воронка, вложенные измерения
111	`private static void addMetricLeaves(FinalDataset ds, int[] rows, int[] rest, FinalGraphFilter segment, Stri…`	сравнение сегмента с остальными по всем метрикам + поправка
150	`private static void addFunnelNode(FinalDataset ds, List<FinalGraphFilter> path, int[] rows, String se…`	узел «Воронка» у сегмента
175	`private static void addSubDimensions(FinalDataset ds, List<FinalGraphFilter> path, int[] rows, int[] …`	вложенные измерения по силе отличия (χ^2)
220	`static void expandMetric(FinalDataset ds, List<FinalGraphFilter> path, int[] rows, FinalColumn c, Str…`	метрика → интервалы гистограммы
252	`static void expandFunnel(FinalDataset ds, List<FinalGraphFilter> path, int[] rows, String parentId, F…`	воронка → этапы с конверсией
316	`static FinalGraphNode dimNode(FinalDataset ds, List<FinalGraphFilter> path, int[] rows, int[] rest, F…`	узел-измерение
363	`private static void describeSegmentDifference(FinalGraphNode n, FinalColumn c, int[] rows, int[] rest)`	чем сегмент отличается по измерению
379	`static FinalGraphNode metricNode(FinalDataset ds, List<FinalGraphFilter> path, int[] rows, FinalColum…`	узел-метрика со статистикой
441	`private static FinalGraphNode segmentNode(FinalDataset ds, List<FinalGraphFilter> path, int[] parentR…`	узел-сегмент с долей и интервалом Уилсона
479	`private static void addMetricEffects(FinalDataset ds, FinalGraphNode n, FinalColumn c, String value, int[] …`	связи «сегмент влияет на метрику»
519	`private static FinalGraphEffect effect(FinalColumn c, String value, FinalColumn metric, FinalWelchTest w)`	одна такая связь
533	`private static FinalGraphNode metricLeaf(String pathKey, MetricTest mt, FinalGraphFilter segment, String pa…`	лист «метрика сегмента» — описывается полиморфно

6.7.4 FinalInsightFinder `service/FinalInsightFinder.java · 167 строк`

Final-класс

```
public final class FinalInsightFinder
```

«Ключевые находки»: сильные сегменты, отвал воронки, сильные связи, тренды, проблемы качества.

> **Почему такой вид.** Final-утилита. Внутри статический вложенный класс SegmentFinding вместо безымянного Object[].

Стр.	Объявление	Что делает
24	`private FinalInsightFinder()`	конструктор
42	`static List<FinalGraphInsight> insights(FinalDataset ds, int[] all, List<FinalGraphLink> pairs)`	собрать и отсортировать все находки
52	`private static void addSegmentInsights(FinalDataset ds, int[] all, List<FinalGraphInsight> out)`	сегменты, сильно отличающиеся по KPI
84	`private static FinalGraphInsight segmentInsight(SegmentFinding f)`	текст находки — через полиморфный insightText
95	`private static void addFunnelInsight(FinalDataset ds, int[] all, List<FinalGraphInsight> out)`	самый большой отвал воронки
114	`private static void addLinkInsights(FinalDataset ds, List<FinalGraphLink> pairs, List<FinalGraphIn…`	самые сильные связи
134	`private static void addTrendInsights(FinalDataset ds, List<FinalGraphInsight> out)`	значимые тренды KPI во времени
154	`private static void addQualityInsights(FinalDataset ds, List<FinalGraphInsight> out)`	колонки с большим числом пропусков

6.7.5 FinalGraphPaths `service/FinalGraphPaths.java · 121 строк`

Final-класс

```
public final class FinalGraphPaths
```

Мелкие общие функции графа: id узлов, пути, воронка, сравнение сегмента по χ^2.

> **Почему такой вид.** Final-утилита; подключается в соседние классы через import static.

Стр.	Объявление	Что делает
21	`private FinalGraphPaths()`	конструктор
24	`public static boolean derivedPair(FinalDataset ds, String a, String b)`	колонки посчитаны одна из другой?
29	`public static FinalChiSquareTest segmentVsRest(FinalColumn d, int[] segment, int[] rest)`	χ^2: сегмент против остальных
45	`public static int[] funnelReached(FinalColumn funnel, int[] rows)`	сколько дошло до каждого этапа
65	`public static int worstStep(int[] reached)`	этап с самой низкой конверсией

79	`static Map<String, Integer> countByLevel(FinalColumn c, int[] rows)`	частоты значений
87	`static int sum(Map<String, Integer> counts)`	сумма частот
93	`static List<String> displayOrder(FinalDataset ds, FinalColumn c, Map<String, Integer> counts)`	порядок показа значений
101	`public static String varId(FinalColumn c)`	id узла переменной
105	`public static String pathKey(List<FinalGraphFilter> path)`	ключ пути
109	`public static String segmentId(List<FinalGraphFilter> path)`	id узла сегмента
113	`public static FinalGraphRef ref(String kind, List<FinalGraphFilter> path, String column, String nodeId)`	ссылка для раскрытия узла

6.8. Пакет ai — ИИ-аналитик

6.8.1 InterfaceAiTool `ai/InterfaceAiTool.java · 12 строк`

Интерфейс

`public interface InterfaceAiTool`

Функциональный интерфейс ИИ-инструмента: выполнить на наборе данных с аргументами от модели.

> **Почему такой вид.** Интерфейс с одним методом (@FunctionalInterface), поэтому реализации пишутся лямбдами.

ai/InterfaceAiTool.java — файл целиком

```
1  package com.analytics.brain.ai;
2
3  import com.analytics.brain.data.FinalDataset;
4  import com.fasterxml.jackson.databind.JsonNode;
5
6  import java.util.function.BiConsumer;
7
8  @FunctionalInterface
9  public interface InterfaceAiTool {
10
11     String run(FinalDataset ds, JsonNode args, BiConsumer<String, Object> emit) throws
   Exception;
12 }
```

6.8.2 ServiceAiTools `ai/ServiceAiTools.java · 311 строк`

Spring-сервис

`public class ServiceAiTools`

17 инструментов ИИ-аналитика в словаре «имя → инструмент» и общие таблицы для отчётов и слайдов.

> **Почему такой вид.** Spring-сервис. Словарь заполняется в конструкторе методом registerTools() — это полиморфизм через функциональный интерфейс вместо длинного switch.

Стр.	Объявление	Что делает
39	`public ServiceAiTools(ServiceBrainGraph brain, ServiceAiClient client, ServiceAiDocuments documents)`	конструктор
46	`String run(FinalDataset ds, String name, JsonNode args, BiConsumer<String, Object> emit) throws Excep…`	найти инструмент в словаре и выполнить
53	`private void registerTools()`	17 лямбд и ссылок на методы
90	`private static String show(Map<String, Object> result, BiConsumer<String, Object> emit) throws …`	отправить таблицу в браузер, модели — сжатую
95	`private static String showWithChart(Map<String, Object> result, BiConsumer<String, Object> emit…`	таблица и график отдельными карточками
104	`private Map<String, Object> correlations(FinalDataset ds, String column, int top)`	сильнейшие связи из обзорного графа

130	`private String generateImage(JsonNode args, BiConsumer<String, Object> emit) throws Exception`	картинка
140	`private String createReport(FinalDataset ds, JsonNode args, BiConsumer<String, Object> emit) throws E…`	отчёт по ГОСТ
147	`private String createPresentation(FinalDataset ds, JsonNode args, BiConsumer<String, Object> emit) th…`	презентация
160	`private String addColumn(FinalDataset ds, JsonNode args, BiConsumer<String, Object> emit) throws Exce…`	колонка по формуле
170	`private String filterDataset(FinalDataset ds, JsonNode args, BiConsumer<String, Object> emit) throws …`	срез данных
182	`private String graphAction(JsonNode args, BiConsumer<String, Object> emit) throws Exception`	подсветить узлы на графе
192	`public static Map<String, Object> table(FinalDataset ds, JsonNode spec) throws Exception`	static: таблица по короткому описанию (для отчётов и /analysis/{kind})
233	`public static List<FinalCondition> conds(JsonNode args) throws Exception`	фильтры из аргументов
239	`public static List<String> strings(JsonNode array)`	массив строк из JSON
246	`private static String forModel(Map<String, Object> result) throws Exception`	сжатая версия результата для модели
260	`public static String stepLabel(String name, JsonNode a)`	подпись шага в чате
307	`private static String join(JsonNode array)`	склеить имена

6.8.3 ServiceAiChat `ai/ServiceAiChat.java · 228 строк`

Spring-сервис

`public class ServiceAiChat`

Диалог с моделью: собирает сообщения, в цикле вызывает модель и инструменты, отправляет события в браузер.

> **Почему такой вид.** Spring-сервис; зависимости приходят через конструктор и хранятся в private final полях.

Стр.	Объявление	Что делает
37	`public ServiceAiChat(ServiceBrainGraph brain, ServiceAiClient client, ServiceAiTools tools) throws IOException`	конструктор
52	`private static String resource(String name) throws IOException`	прочитать файл из resources
58	`public void chat(FinalChatRequest request, BiConsumer<String, Object> emit) throws Exception`	цикл «модель → инструмент → модель», не больше 10 шагов
109	`private ArrayNode buildMessages(FinalChatRequest request, FinalDataset ds, BiConsumer<String, Object>…`	системная инструкция + описание данных + история

139	`private String runCall(FinalDataset ds, JsonNode call, BiConsumer<String, Object> emit) throws Exception`	выполнить один вызов инструмента
158	`private String describeImages(String question, List<String> pictures, BiConsumer<String, Object>…`	описать картинки через vision-модель
186	`String context(FinalDataset ds)`	описание набора данных для модели

6.8.4 ServiceAiClient `ai/ServiceAiClient.java · 332 строк`

Spring-сервис

`public class ServiceAiClient`

HTTP-клиент модели: настройки, список моделей, запрос к chat/completions с понижением параметров при ошибках, генерация картинок.

> **Почему такой вид.** Spring-сервис. Общий ObjectMapper JSON — public static final, им пользуются другие классы пакета.

Стр.	Объявление	Что делает
64	`public ServiceAiClient() throws IOException`	конструктор
83	`private void fromEnv(String variable, String property)`	взять настройку из переменной окружения
88	`String setting(String name, String fallback)`	значение настройки
92	`public Map<String, Object> config()`	настройки (ключ замаскирован)
112	`public synchronized Map<String, Object> saveConfig(Map<String, String> input) throws IOException`	сохранить настройки
129	`public List<String> models() throws Exception`	модели провайдера
144	`String resolve(String wanted, boolean vision)`	выбрать модель
166	`String imageModel()`	модель для картинок
186	`private static String normalize(String s)`	нормализовать имя модели
200	`JsonNode chatCompletion(ObjectNode body) throws Exception`	запрос к модели с повторами
248	`public Object[] image(String id)`	отдать картинку
252	`public String generateImage(String prompt, String aspect, String theme) throws Exception`	сгенерировать картинку
303	`private Object[] readPicture(JsonNode data) throws Exception`	достать картинку из ответа провайдера
327	`public static String snippet(String s)`	static: короткий фрагмент текста ошибки

6.8.5 ServiceAiDocuments `ai/ServiceAiDocuments.java · 215 строк`

Spring-сервис

`public class ServiceAiDocuments`

Сборка презентаций и отчётов по ГОСТ из описания, которое пишет модель; все таблицы пересчитывает сервер.

> **Почему такой вид.** Spring-сервис.

Стр.	Объявление	Что делает
30	`public ServiceAiDocuments(ServiceAiClient client)`	конструктор
34	`Map<String, Object> buildDeck(FinalDataset ds, JsonNode spec)`	презентация
52	`private Map<String, Object> buildSlide(FinalDataset ds, JsonNode s)`	один слайд
81	`private void addImages(List<Map<String, Object>> slides, String theme)`	картинки для слайдов
111	`List<Map<String, Object>> kpis(FinalDataset ds, JsonNode list) throws Exception`	плитки KPI
141	`Map<String, Object> buildReport(FinalDataset ds, JsonNode spec) throws Exception`	отчёт
180	`private Map<String, Object> buildBlock(FinalDataset ds, JsonNode b, String sectionTitle, List<Stri…`	блок отчёта
201	`private static Map<String, Object> dataDescription(FinalDataset ds)`	описание данных для отчёта

6.8.6 FinalChatRequest `ai/FinalChatRequest.java · 13 строк`

Final-класс

`public final class FinalChatRequest`

Тело запроса чата: id набора, модель, история сообщений, выбранный узел.

> **Почему такой вид.** Final-класс данных (DTO).

ai/FinalChatRequest.java — файл целиком

```
1  package com.analytics.brain.ai;
2
3  import java.util.ArrayList;
4  import java.util.List;
5  import java.util.Map;
6
7  public final class FinalChatRequest {
8
9      public String datasetId;
10     public String model;
11     public List<Map<String, Object>> messages = new ArrayList<>();
12     public Map<String, Object> context;
13 }
```

6.9. Пакет api — REST-контроллеры

6.9.1 AbstractApiController `api/AbstractApiController.java · 43 строк`

Абстрактный класс наследники: `ControllerAiApi`, `ControllerBrainApi`

```
public abstract class AbstractApiController
```

Общий предок двух контроллеров: ссылка на ServiceBrainGraph, поиск набора по id и обработчики ошибок (404 и 400).

> **Почему такой вид.** Абстрактный: у каждого контроллера своя подсказка к «не найдено» и свой текст неизвестной ошибки — это два абстрактных метода.

api/AbstractApiController.java — файл целиком

```
1  package com.analytics.brain.api;
2
3  import com.analytics.brain.data.FinalDataset;
4  import com.analytics.brain.exception.ExceptionDatasetNotFound;
5  import com.analytics.brain.service.ServiceBrainGraph;
6  import com.fasterxml.jackson.databind.ObjectMapper;
7  import org.springframework.http.ResponseEntity;
8  import org.springframework.web.bind.annotation.ExceptionHandler;
9
10 import java.util.Map;
11 import java.util.NoSuchElementException;
12
13 public abstract class AbstractApiController {
14
15     protected static final ObjectMapper JSON = new ObjectMapper();
16
17     protected final ServiceBrainGraph brain;
18
19     protected AbstractApiController(ServiceBrainGraph brain) {
20         this.brain = brain;
21     }
22
23     protected final FinalDataset dataset(String id) {
24         FinalDataset d = brain.store.get(id);
25         if (d == null) throw new ExceptionDatasetNotFound("Набор данных не найден: " + id +
notFoundHint());
26         return d;
27     }
28
29     protected abstract String notFoundHint();
30
31     protected abstract String unknownErrorText(Exception e);
32
33     @ExceptionHandler(NoSuchElementException.class)
34     public ResponseEntity<Map<String, Object>> notFound(Exception e) {
35         return ResponseEntity.status(404).body(Map.of("error", e.getMessage()));
36     }
37
38     @ExceptionHandler(Exception.class)
39     public ResponseEntity<Map<String, Object>> error(Exception e) {
40         String message = e.getMessage() == null ? unknownErrorText(e) : e.getMessage();
41         return ResponseEntity.badRequest().body(Map.of("error", message));
42     }
43 }
```

6.9.2 ControllerBrainApi `api/ControllerBrainApi.java · 177 строк`

REST-контроллер наследует `AbstractApiController`

```
public class ControllerBrainApi extends AbstractApiController
```

REST API данных и графа: загрузка, список, обзор, раскрытие, обновление, дописывание строк, удаление, демо-данные.

> **Почему такой вид.** @RestController, наследник AbstractApiController. Конструктор принимает ServiceBrainGraph и передаёт его в super(brain).

Стр.	Объявление	Что делает
38	`public ControllerBrainApi(ServiceBrainGraph brain)`	конструктор
43	`protected String notFoundHint()`	@Override
48	`protected String unknownErrorText(Exception e)`	@Override
53	`public Map<String, Object> create(@RequestBody FinalCreateDatasetRequest r) throws Exception`	POST /datasets
83	`public List<Map<String, Object>> list()`	GET /datasets
90	`public Map<String, Object> meta(@PathVariable String id)`	GET /datasets/{id}
95	`public FinalGraphPayload overview(@PathVariable String id)`	GET /datasets/{id}/overview
100	`public FinalGraphPayload expand(@PathVariable String id, @RequestBody FinalGraphRef ref)`	POST /datasets/{id}/expand
105	`public Map<String, Object> refresh(@PathVariable String id) throws Exception`	POST /datasets/{id}/refresh
119	`public Map<String, Object> append(@PathVariable String id, @RequestBody FinalCreateDatasetRequest r) …`	POST /datasets/{id}/append
155	`public Map<String, Object> delete(@PathVariable String id)`	DELETE /datasets/{id}
161	`public ResponseEntity<String> demo(@RequestParam(defaultValue = "1000") int rows,`	GET /demo/users
173	`private static String shortName(String url)`	короткое имя по ссылке

6.9.3 ControllerAiApi `api/ControllerAiApi.java · 139 строк`

REST-контроллер наследует `AbstractApiController`

```
public class ControllerAiApi extends AbstractApiController
```

REST API ИИ и аналитики: настройки, модели, чат (поток SSE), картинки, сводные, графики, формулы, анализы.

> **Почему такой вид.** @RestController, наследник AbstractApiController.

Стр.	Объявление	Что делает

38	`public ControllerAiApi(ServiceBrainGraph brain, ServiceAiChat ai, ServiceAiClient client)`	конструктор
45	`protected String notFoundHint()`	@Override
50	`protected String unknownErrorText(Exception e)`	@Override
55	`public Map<String, Object> config()`	GET /ai/config
60	`public Map<String, Object> saveConfig(@RequestBody Map<String, String> input) throws Exception`	POST /ai/config
65	`public Map<String, Object> models() throws Exception`	GET /ai/models
70	`public ResponseEntity<byte[]> image(@PathVariable String id)`	GET /ai/images/{id}
80	`public Map<String, Object> generateImage(@RequestBody Map<String, String> b) throws Exception`	POST /ai/image
86	`public SseEmitter chat(@RequestBody FinalChatRequest request)`	POST /ai/chat — поток событий
106	`private static void send(SseEmitter emitter, String event, Object data)`	отправить событие
115	`public Map<String, Object> pivot(@PathVariable String id, @RequestBody FinalPivotSpec spec)`	POST /datasets/{id}/pivot
120	`public Map<String, Object> chart(@PathVariable String id, @RequestBody FinalChartSpec spec)`	POST /datasets/{id}/chart
125	`public Map<String, Object> column(@PathVariable String id, @RequestBody Map<String, String> b)`	POST /datasets/{id}/column
133	`public Map<String, Object> analysis(@PathVariable String id, @PathVariable String kind,`	POST /datasets/{id}/analysis/{kind}

6.9.4 FinalCreateDatasetRequest `api/FinalCreateDatasetRequest.java · 14 строк`

Final-класс

`public final class FinalCreateDatasetRequest`

Тело запроса загрузки: заполняется одно из csv, data или url.

> **Почему такой вид.** Final-класс данных (DTO).

api/FinalCreateDatasetRequest.java — файл целиком

```java
package com.analytics.brain.api;

import com.fasterxml.jackson.databind.JsonNode;

import java.util.Map;

public final class FinalCreateDatasetRequest {

    public String name;
    public String csv;
    public String url;
    public JsonNode data;
    public Map<String, String> headers;
}
```

6.10. Пакет text — форматы

6.10.1 FinalFormatRu `text/FinalFormatRu.java · 137 строк`

Final-класс

```
public final class FinalFormatRu
```

Русские форматы: числа с неразрывным пробелом и запятой, проценты, знаки, p-значения, склонения.

> **Почему такой вид.** Final-утилита; подключается через import static.

Стр.	Объявление	Что делает
9	`private FinalFormatRu()`	конструктор
12	`public static String fmt(double v)`	число: 1 250 / 12,5 / 0,034
32	`public static String fmtInt(long v)`	целое с пробелами
36	`public static String sgn(double v)`	со знаком
40	`public static String pct(double v)`	процент
49	`public static String pctAbs(double v)`	процент без знака
53	`public static String sgnPct(double v)`	процент со знаком
57	`public static String sgnPP(double v)`	процентные пункты
63	`public static String fmtP(double p)`	p-значение
68	`public static String fmtPq(double p)`	«< 0,001» или «= 0,03»
72	`public static String levelRuF(String level)`	сила по-русски (жен. род)
85	`public static String levelRu(String level)`	сила по-русски
98	`public static int levelRank(String level)`	сила числом
105	`public static String plural(int n, String one, String few, String many)`	склонение
114	`public static String granularityRu(String granularity)`	гранулярность по-русски
128	`public static String purchaseVerb(String columnName)`	«купили», «обращались»…

6.11. Пакет exception — исключения

6.11.1 ExceptionAnalitikData `exception/ExceptionAnalitikData.java · 8 строк`

Исключение наследует `IllegalArgumentException`

```
public class ExceptionAnalitikData extends IllegalArgumentException
```

Ошибка во входных данных: нет колонки, мало строк, неверная формула. Контроллер превращает её в ответ 400 с текстом.

> **Почему такой вид.** Наследник IllegalArgumentException: весь старый код, который ловит IllegalArgumentException, продолжает работать.

exception/ExceptionAnalitikData.java — файл целиком

```
1  package com.analytics.brain.exception;
2
3  public class ExceptionAnalitikData extends IllegalArgumentException {
4
5      public ExceptionAnalitikData(String message) {
6          super(message);
7      }
8  }
```

6.11.2 ExceptionDatasetNotFound `exception/ExceptionDatasetNotFound.java · 10 строк`

Исключение наследует `NoSuchElementException`

```
public class ExceptionDatasetNotFound extends NoSuchElementException
```

Набор данных не найден по id. Обработчик NoSuchElementException в контроллере отдаёт 404.

> **Почему такой вид.** Наследник NoSuchElementException.

exception/ExceptionDatasetNotFound.java — файл целиком

```
1  package com.analytics.brain.exception;
2
3  import java.util.NoSuchElementException;
4
5  public class ExceptionDatasetNotFound extends NoSuchElementException {
6
7      public ExceptionDatasetNotFound(String message) {
8          super(message);
9      }
10 }
```

7 Математика и алгоритмы простыми словами

Для каждого алгоритма: что он делает, формула, шаги и где это в коде. Все формулы тестов написаны вручную; из библиотеки Apache Commons Math взяты только функции распределений (t, χ^2, F, нормальное), решение МНК и QR-разложение.

7.1. p-значение и уровень 0,05

Каждый тест отвечает на вопрос: насколько вероятно увидеть такую же или ещё более сильную связь, если на самом деле связи нет. Эта вероятность — p. Если p < 0,05, связь называют значимой. Константа `ALPHA = 0.05` лежит в `AbstractAnalitikMathematicsTest`.

7.2. Корреляция Спирмена — FinalSpearmanTest

Идея. Заменить числа их рангами (1 — самое маленькое) и посчитать обычную корреляцию Пирсона по рангам. Выбросы не мешают, и ловится любая монотонная связь, не только прямая линия.

Шаги. 1) Выбросить пары с пропуском (`bothPresent`). 2) Ранги: сортируем индексы, одинаковым значениям даём средний ранг (`ranks`). 3) ρ = Пирсон(ранги x, ранги y). 4) $t = \rho \cdot \sqrt{(n-2)/(1-\rho^2)}$, p — по распределению Стьюдента с n−2 степенями свободы. 5) Интервал: $z = \frac{1}{2} \cdot \ln((1+\rho)/(1-\rho))$ (преобразование Фишера), $z \pm 1{,}96 \cdot \sqrt{1{,}06/(n-3)}$, обратно через tanh.

Ранги со средним рангом для одинаковых значений · mathematics/FinalAnalitikMathematics.java, строки 128–144

```java
public static double[] ranks(double[] x) {
    int n = x.length;
    Integer[] order = new Integer[n];
    for (int i = 0; i < n; i++) order[i] = i;
    Arrays.sort(order, Comparator.comparingDouble(i -> x[i]));

    double[] rank = new double[n];
    int i = 0;
    while (i < n) {
        int j = i;
        while (j + 1 < n && x[order[j + 1]] == x[order[i]]) j++;
        double average = (i + j) / 2.0 + 1;
        for (int k = i; k <= j; k++) rank[order[k]] = average;
        i = j + 1;
    }
    return rank;
}
```

7.3. χ^2 и V Крамера — FinalChiSquareTest

Идея. Если две категории независимы, в клетке таблицы «город × подписка» должно быть примерно E = (итог строки · итог столбца) / n наблюдений. $\chi^2 = \Sigma\,(O - E)^2 / E$ показывает, насколько реальность далека от независимости. Степени свободы (r−1)(k−1). Сила связи — V Крамера = $\sqrt{\chi^2/n\,/\,(\min(r,k)-1)}$; мы берём исправленный V Бергсмы, потому что обычный V завышен на маленьких выборках.

Исправленный V Крамера · mathematics/FinalChiSquareTest.java, строки 68–75

```java
private double correctedCramersV(double chi2, int r, int k) {
    double phi2 = chi2 / n;
    double phi2c = Math.max(0, phi2 - (double) (k - 1) * (r - 1) / (n - 1));
    double rc = r - (double) (r - 1) * (r - 1) / (n - 1);
    double kc = k - (double) (k - 1) * (k - 1) / (n - 1);
    double denom = Math.min(kc - 1, rc - 1);
    return denom <= 0 ? 0 : Math.sqrt(phi2c / denom);
}
```

7.4. ANOVA и Краскел–Уоллис — FinalAnovaTest

Идея. Разброс всех значений делится на разброс между группами (SSB) и внутри групп (SSW). F = (SSB/(k−1)) / (SSW/(n−k)). η^2 = SSB / SST — доля разброса, которую объясняет группа; на граф идёт $\eta = \sqrt{\eta^2}$. ω^2 — то же, но менее завышенное. Краскел–Уоллис — та же идея по рангам: $H = 12/(n(n+1))\cdot\Sigma\, R^2_i/n_i - 3(n+1)$, нормальность данных не нужна.

7.5. t-тест Уэлча и g Хеджеса — FinalWelchTest

$t = (\bar{x}_a - \bar{x}_\beta) / \sqrt{(s_a^2/n_a + s_\beta^2/n_\beta)}$. В отличие от классического t-теста, дисперсии групп могут различаться. Степени свободы — по формуле Уэлча–Саттертуэйта. g Хеджеса — разница средних в стандартных отклонениях (d Коэна), умноженная на поправку $1 - 3/(4(n_a+n_\beta) - 9)$.

Уэлч и Хеджес · mathematics/FinalWelchTest.java, строки 25–47

```java
double va = FinalAnalitikMathematics.variance(a);
double vb = FinalAnalitikMathematics.variance(b);
double se2 = va / a.length + vb / b.length;
if (se2 == 0) {
    p = meanA == meanB ? 1 : 0;
} else {
    stat = (meanA - meanB) / Math.sqrt(se2);
    double partA = Math.pow(va / a.length, 2) / (a.length - 1);
    double partB = Math.pow(vb / b.length, 2) / (b.length - 1);
    df1 = se2 * se2 / (partA + partB);
    p = FinalProbabilityDistributions.tP(stat, df1);
}

double pooledSd = Math.sqrt(((a.length - 1) * va + (b.length - 1) * vb) / (a.length + b.length - 2));
double d = pooledSd == 0 ? 0 : (meanA - meanB) / pooledSd;
double correction = 1 - 3.0 / (4.0 * (a.length + b.length) - 9);
effect = d * correction;

double seG = Math.sqrt((double) (a.length + b.length) / (a.length * b.length)
        + effect * effect / (2.0 * (a.length + b.length)));
lo = effect - 1.96 * seG;
hi = effect + 1.96 * seG;
classifyEffect();
```

7.6. z-тест двух долей и h Коэна — FinalTwoProportionTest

Объединённая доля $p = (k_a+k_\beta)/(n_a+n_\beta)$, $SE = \sqrt{(p(1-p)(1/n_a+1/n_\beta))}$, $z = (p_a - p_\beta)/SE$. $h = 2\cdot\arcsin\sqrt{p_a} - 2\cdot\arcsin\sqrt{p_\beta}$: арксинус нужен, потому что разница 1% → 5% важнее, чем 50% → 54%.

7.7. Интервал Уилсона — FinalAnalitikMathematics.wilson

Школьная формула p ± 1,96·√(p(1−p)/n) на маленьких n выходит за 0 и 1. Уилсон сдвигает центр к 50%: центр = (p + z²/2n)/(1 + z²/n), полуширина = z·√(p(1−p)/n + z²/4n²)/(1 + z²/n).

Интервал Уилсона · mathematics/FinalAnalitikMathematics.java, строки 109–117

```java
public static double[] wilson(int k, int n) {
    if (n == 0) return new double[]{0, 0};
    double z = Z_95;
    double p = (double) k / n;
    double denom = 1 + z * z / n;
    double center = (p + z * z / (2.0 * n)) / denom;
    double half = z * Math.sqrt(p * (1 - p) / n + z * z / (4.0 * n * n)) / denom;
    return new double[]{Math.max(0, center - half), Math.min(1, center + half)};
}
```

7.8. Поправка Бенджамини–Хохберга — FinalMultipleComparisons

Зачем. В демо-наборе 22 переменные — это 231 пара. На случайных данных примерно 231 × 0,05 ≈ 12 пар покажутся «значимыми». Поправка оставляет только такие связи, среди которых ложных в среднем не больше 5%.

Шаги. 1) Отсортировать p по возрастанию, ранг 1 — самое маленькое. 2) q = p · m / ранг, где m — число проверок. 3) Пройти от последнего ранга к первому и брать минимум — так q не убывает с рангом. 4) Связь значима, если q < 0,05. Пример: для 20 проверок с p = 0,001; 0,003; 0,005; 0,012… без поправки значимы 9, с поправкой — 3.

7.9. Шкала силы Коэна — EnumEffectScale

Шкала	Для чего	слабая	средняя	сильная
CORRELATION	ρ, V, η, β	0,1	0,3	0,5
DIFFERENCE	g Хеджеса, h Коэна	0,2	0,5	0,8

Второй фильтр после поправки: при огромной выборке даже ничтожная связь станет значимой, поэтому на граф попадает только то, что и значимо, и не слабее «слабой».

7.10. Регрессия — FinalDriversAnalysis

Идея. Парная связь может обманывать (лояльность и выручка обе растут от подписки). Регрессия МНК оценивает вклад каждого фактора при остальных фиксированных. **Шаги:** убрать «утечки» (колонки, посчитанные из цели, и почти равные ей: связь ≥ 0,9); взять полные строки; числа перевести в z-оценки, категории — в дамми 0/1 (самое частое значение — база); жадно убрать коллинеарные факторы: берём факторы по убыванию связи с целью и оставляем очередной, только если QR-разложение говорит, что матрица невырождена; факторов не больше n/15; решить МНК; для каждого коэффициента t = β/SE и p; поправка Бенджамини–Хохберга по всем коэффициентам.

Жадный отбор факторов с QR-проверкой · analysis/FinalDriversAnalysis.java, строки 233–264

```java
private static List<Factor> dropCollinear(List<Factor> factors, double[] yv, int n, List<String> collinear) {
    Integer[] order = new Integer[factors.size()];
    double[] strength = new double[factors.size()];
    for (int j = 0; j < order.length; j++) {
        order[j] = j;
        strength[j] = Math.abs(FinalAnalitikMathematics.pearson(factors.get(j).values, yv));
    }
    Arrays.sort(order, (a, b) -> Double.compare(strength[b], strength[a]));

    List<Integer> keep = new ArrayList<>();
    int limit = Math.max(1, n / ROWS_PER_FACTOR);
    for (int j : order) {
        if (keep.size() >= limit) break;
        List<Integer> trial = new ArrayList<>(keep);
        trial.add(j);
        double[][] a = new double[n][trial.size() + 1];
        for (int k = 0; k < n; k++) {
            a[k][0] = 1;
            for (int q = 0; q < trial.size(); q++) a[k][q + 1] = factors.get(trial.get(q)).values[k];
        }
        boolean ok = new QRDecomposition(new Array2DRowRealMatrix(a, false), 1e-8).getSolver().isNonSingular();
        if (ok) {
            keep.add(j);
        } else {
            collinear.add(factors.get(j).name);
        }
    }

    List<Factor> kept = new ArrayList<>();
    for (int j : keep) kept.add(factors.get(j));
    return kept;
}
```

7.11. Тренд и прогноз — FinalTrendAnalysis

Значения по периодам → прямая y = a + b·x методом наименьших квадратов (b = Sxy/Sxx). Наклон проверяется t-тестом: t = b / √(s²/Sxx). Прогноз на h периодов с 95% интервалом предсказания: $\hat{y} \pm t \cdot \sqrt{(s^2 \cdot (1 + 1/k + (x - \bar{x})^2/Sxx))}$; интервал расширяется по мере удаления от данных.

7.12. Аномалии — FinalGroupAnomalyAnalysis и FinalRecordAnomalyAnalysis

Группы. Робастная z-оценка: z = (x − медиана) / (1,4826 · MAD), где MAD — медиана отклонений от медианы. Обычное среднее и σ ломаются как раз от тех выбросов, которые ищем. |z| ≥ 3 — аномалия, |z| ≥ 2 — внимание. **Записи.** Правило Тьюки: всё, что ниже Q1 − 3·IQR или выше Q3 + 3·IQR.

7.13. ABC и коэффициент Джини

ABC: группы сортируются по убыванию вклада; класс определяется по накопленной доле до группы: меньше 80% — A, меньше 95% — B, иначе C. Джини: $G = 2 \cdot \Sigma(i \cdot x_i)/(n \cdot \Sigma x) - (n+1)/n$ по отсортированной выручке; 0 — все приносят поровну, ближе к 1 — почти всё приносят немногие.

7.14. Гистограмма Фридмана–Диакониса

Ширина столбика h = 2·IQR·n^(−1/3), затем округление до «красивого» шага (1, 2, 2,5, 5, 10 × 10ⁿ) и ограничение 4–8 столбиков. Если разных значений не больше 8, каждое значение — свой столбик.

7.15. Рекурсивный спуск — FinalFormulaParser

Каждому уровню приоритета операций соответствует метод, и каждый метод вызывает следующий, более сильный: `or → and → comparison → sum → product → unary → atom`. Поэтому в формуле `1 + 2 * 3` сначала «соберётся» умножение: `sum` попросит у `product` правый операнд, а тот съест `2 * 3` целиком. Скобки в `atom` снова вызывают `or` — отсюда «рекурсивный». Результат — дерево объектов; вычисление дерева для строки — тоже рекурсия: каждый узел вызывает `evaluate` у своих детей.

Уровень «+ и −» · formula/FinalFormulaParser.java, строки 63–74

```java
private InterfaceFormulaNode sum() {
    InterfaceFormulaNode node = product();
    while (true) {
        if (eat("+")) {
            node = new FinalBinaryNode(EnumFormulaOperator.PLUS, node, product());
        } else if (eat("-")) {
            node = new FinalBinaryNode(EnumFormulaOperator.MINUS, node, product());
        } else {
            return node;
        }
    }
}
```

Уровень «*, / и %» · formula/FinalFormulaParser.java, строки 76–89

```java
private InterfaceFormulaNode product() {
    InterfaceFormulaNode node = unary();
    while (true) {
        if (eat("*")) {
            node = new FinalBinaryNode(EnumFormulaOperator.MULTIPLY, node, unary());
        } else if (eat("/")) {
            node = new FinalBinaryNode(EnumFormulaOperator.DIVIDE, node, unary());
        } else if (eat("%")) {
            node = new FinalBinaryNode(EnumFormulaOperator.MODULO, node, unary());
        } else {
            return node;
        }
    }
}
```

7.16. Определение типа колонки — FinalDataLoader.infer

Правила проверяются строго по порядку: все пустые → текст; ≥ 95% значений читаются как даты → дата; ≥ 95% — как числа → число (или ID, если в имени id, номер, код и все значения разные целые); много разных строк или имя вроде e-mail → текст или ID; иначе категория (14 самых частых значений + «Другое»). Число с 8–92% нулей — «счётчик»: для него сегменты сравниваются z-тестом долей.

Порядок правил · data/FinalDataLoader.java, строки 163–215

```java
public static FinalColumn infer(String name, String[] raw, int n) {
    FinalColumn column = new FinalColumn(name);
    int filled = 0;
    int numbers = 0;
    int dates = 0;
    Set<String> distinct = new HashSet<>();
    double[] num = new double[n];
    long[] days = new long[n];

    for (int i = 0; i < n; i++) {
        num[i] = Double.NaN;
        days[i] = Long.MIN_VALUE;
        if (raw[i] == null) continue;
        filled++;
        distinct.add(raw[i]);

        double d = parseNum(raw[i]);
        if (!Double.isNaN(d)) {
            num[i] = d;
            numbers++;
        }
        LocalDate date = parseDate(raw[i]);
        if (date != null) {
            days[i] = date.toEpochDay();
            dates++;
        }
    }
    column.missing = n - filled;
    column.unique = distinct.size();

    String trimmed = name.trim();
    boolean looksLikeId = ID_NAME.matcher(trimmed).find()
            || (SOFT_ID_NAME.matcher(trimmed).find() && distinct.size() > 30);

    if (filled == 0) {
        column.type = EnumColumnType.TEXT;
        return column;
    }
    if (dates >= 0.95 * filled && !looksLikeId) {
        makeDate(column, days, n);
        return column;
    }
    if (numbers >= 0.95 * filled) {
        makeNumeric(column, name, raw, num, filled, looksLikeId);
        return column;
    }
    if (looksLikeId || (column.unique > 60 && column.unique > filled * 0.5)) {
        column.type = looksLikeId ? EnumColumnType.ID : EnumColumnType.TEXT;
        return column;
    }
    makeCategorical(column, raw, n);
    return column;
}
```

8 Те же идеи в C++

Если спросят «а как это было бы на плюсах», вот соответствие конструкций.

В проекте (Java)	В C++	Разница
`interface InterfaceFormulaNode`	класс только с `virtual … = 0`	отдельного слова interface в C++ нет
`abstract class`, `abstract`-метод	класс с хотя бы одной `= 0` функцией	в Java можно пометить abstract класс даже без абстрактных методов
`final class`	`class X final`	одинаково
`final`-метод	`void f() override final;`	в C++ final только у виртуальных методов
`final`-поле	`const`-поле	в C++ ещё есть const-методы, в Java нет
`@Override`	`override`	обе проверяют сигнатуру при компиляции
все методы виртуальные	только помеченные `virtual`	в Java не нужно слово virtual
`extends` одного класса + `implements` многих интерфейсов	множественное наследование классов	в Java нет ромба
`super(…)` в конструкторе	`: Base(…)` в списке инициализации	порядок один: сначала родитель
enum с полями и методами	`enum class` без методов	поведение пришлось бы писать отдельными функциями
сборщик мусора	деструкторы, RAII, `unique_ptr`	в Java нет деструкторов
статический утилитный класс	`namespace` со свободными функциями	в Java функция обязана жить в классе
лямбда для `InterfaceAiTool`	`std::function` + лямбда	идея та же

Иерархия тестов на C++ выглядела бы так:

Эквивалент на C++ (для сравнения, в проекте его нет)

```cpp
class SignificanceTest {                       // InterfaceSignificanceTest
public:
    virtual ~SignificanceTest() = default;
    virtual double pValue() const = 0;
    virtual void acceptQValue(double q) = 0;
};

class AnalitikMathematicsTest : public SignificanceTest {   // abstract
public:
    double p = 1, q = 1, effect = 0;
    double pValue() const override final { return p; }
    void acceptQValue(double v) override final { q = v; }
    bool significant() const { return q < 0.05 && level != "none"; }
protected:
    std::string level = "none";
    virtual EffectScale scale() const = 0;
};

class CorrelationTest : public AnalitikMathematicsTest {
protected:
    EffectScale scale() const override final { return EffectScale::Correlation; }
};

class SpearmanTest final : public CorrelationTest {
public:
    SpearmanTest(const std::vector<double>& a, const std::vector<double>& b);
};
```

9 Вопросы и ответы для защиты

Где в проекте классы?

Каждый .java-файл в папке brain и её подпапках — это один класс, интерфейс или перечисление; имя файла совпадает с именем типа. Первое слово имени говорит, какого он вида: Interface, Abstract, Final, Enum, Exception, Service, Controller.

Где наследование?

В строках объявления после слова extends. Главный пример — статистические тесты: FinalWelchTest extends AbstractSegmentComparisonTest extends AbstractAnalitikMathematicsTest. Ещё анализы (FinalTrendAnalysis extends AbstractAnalitikAnalysis), контроллеры (ControllerBrainApi extends AbstractApiController) и исключения.

Зачем здесь final у класса?

Чтобы от класса нельзя было наследоваться. Конечный тест или анализ — законченная формула, её не должны тихо подменить в наследнике. У утилит (FinalAnalitikMathematics) все методы статические, наследовать там нечего.

Зачем final у метода, например run()?

Чтобы наследник не мог его переопределить. run() задаёт порядок шагов для всех анализов; наследник меняет только compute(). Если бы run() не был final, кто-то мог бы сломать общий формат результата.

Зачем final у поля?

Значение присваивается один раз. Имя колонки, левый и правый узел формулы, зависимости сервисов не должны меняться после создания объекта.

Зачем static final?

Это константа: одна на весь класс и неизменяемая. Например ALPHA = 0.05, Z_95 = 1.959964, TUKEY_K = 3.

Чем абстрактный класс отличается от интерфейса?

Абстрактный класс может хранить поля, иметь конструктор и готовые методы; наследовать можно только один класс. Интерфейс — только контракт из методов (плюс default/static), реализовать можно сколько угодно интерфейсов. Поэтому общие поля тестов лежат в абстрактном классе, а «умение отдать p» — в интерфейсе.

Где полиморфизм?

applyFdr работает со списком любых тестов через интерфейс; run() вызывает compute() конкретного анализа; узел формулы вызывает evaluate() у детей, не зная их классов; у каждой константы EnumFormulaOperator свой apply; metricLeaf вызывает leafSubtitle() — разный текст у Уэлча и z-теста; ИИ-инструменты берутся из словаря и вызываются через интерфейс.

Что такое @Override?

Пометка, что метод переопределяет метод родителя или интерфейса. Компилятор проверит, что такой метод действительно есть, иначе ошибка.

Что делает super(...)?

Вызывает конструктор родителя. Это обязательно первая строка конструктора наследника: сначала строится родительская часть объекта.

Почему нельзя написать new AbstractAnalitikAnalysis(ds)?

Класс абстрактный: у него нет метода compute(), значит, объект был бы недоделанным. Компилятор запрещает.

Почему поля тестов public?

Объекты тестов целиком уходят в браузер как JSON, а библиотека Jackson сериализует публичные поля. Это классы-данные; логика у них в конструкторе и методах.

Что такое шаблонный метод?

Паттерн, при котором базовый класс задаёт порядок шагов в final-методе (run), а отдельные шаги — абстрактные методы, которые пишут наследники (compute).

Что такое фабричный метод?

Статический метод, который сам решает, объект какого подкласса создать, и возвращает общий тип: AbstractSegmentComparisonTest.compare(...) и AbstractAnomalyAnalysis.create(...).

Зачем enum с методами?

Когда вариантов фиксированное число, но у каждого своё поведение: 15 операций формулы, 15 функций, 2 шкалы силы. Каждая константа — отдельный объект со своим телом метода.

Что такое лямбда в registerTools?

Короткая запись объекта, реализующего функциональный интерфейс InterfaceAiTool. (ds, args, emit) -> ... — это реализация его единственного метода run.

Что такое @Service и @RestController?

Аннотации Spring. Объекты таких классов создаёт фреймворк, по одному на приложение, и передаёт в конструкторы других классов (внедрение зависимостей).

Как проверили, что после перестройки ничего не сломалось?

Запустили старую и новую версию сервера и отправили в обе одинаковые 723 запроса: загрузки, обзоры, раскрытия веток, сводные, все анализы, чат с тестовой моделью. Ответы совпали полностью, отличался только номер порта. Отдельно сравнили 80 формул, включая ошибочные, — 0 различий.